

An optimisation approach for supply chain procurement transport operational planning with 3D full-truck loading constraints and returnable transport items

This paper proposes a new mixed integer linear mathematical programming (MILP) model, dubbed 3D-SCPTOP, for procurement transport planning in manufacturing supply chains. The model effectively integrates procurement and transport decisions while accounting for three-dimensional full-truck loading constraints and managing returnable transport items (RTIs). Thus, we optimise the material to procure for an assembly manufacturing site, RTIs closed-loop management, and the involved transport decisions, including the truck loading configuration. Procurement transport decisions are based on the demand of assemblers, the supplier’s inventory levels, and the dimensions of the truck / RTIs. The optimisation model aims to minimise transportation and inventory costs. Because the loading subproblem is NP-hard and is replicated once per loading configuration, the compact model is intractable at industrial sizes. We therefore reformulate it by Dantzig–Wolfe decomposition, taking the complete loaded truck as the unit of decision: a column is a fully specified, geometrically realisable truck load, so every placement, height, crush-limit, payload and full-truckload constraint holds inside it by construction, and the per-stack crush cap is evaluated on the actual product mix rather than on a worst-case bound. Columns are priced on demand by a constraint programming oracle, and the master is solved by a two-level branch and bound over the fleet vector, yielding MATH-3D-SCPTOP: the first level partitions on the total number of trucks, whose cost the relaxation cannot offset, and the second on the per-period dispatch counts inside a fixed total. Its primal heuristic is a flow-first construction that decides shipment quantities and truck counts in an aggregate integer master and then partitions each period’s deliveries into exactly that many legal trucks, so the pool receives both the extreme loads that pricing returns and the balanced loads that no dual vector can produce. We report a lower bound only when a full pricing sweep has proved that no column prices out, and none otherwise; because every placeable load is volumetrically feasible, the certified bound of the volumetric relaxation is a valid lower bound for the 3D problem, and it is the bound the method uses. Columns arrive through a pricing ladder: a heuristic rung run under the current duals serves most pairs, the exact oracle the rest, and the certifying sweep is always exact. One instance is solved to proved optimality, and not by the method: on the five-period instance the volumetric relaxation itself solves to optimality at exactly the cost of our verified 3D plan, which makes that plan optimal by inclusion; no run proves optimality by its own certification, and the method’s attempt to do so on the same instance failed. On the five-period instance where the compact 3D model returns a plan, the reformulation’s plan is 0.10% cheaper than the compact model’s and lies within 0.0076% of the method’s own certified bound; the compact model’s dual bound is not quoted as a bound on the optimum, because that model restricts each direction a priori to three loading configurations and a restriction bounds only itself; from ten periods on the compact model finds no feasible solution within its budget while the reformulation returns verified plans, with gaps of 0.17% and 0.03% to its own certified bound at ten and twelve periods (0.010% to the compact volumetric model’s dual bound at ten), a certified bound 7.4% above the compact model’s own dual bound at twelve, and truck counts proved minimal through twelve periods by a certificate that the region with one truck fewer is infeasible; at twenty periods, twenty-four products and a fleet of twenty-one it certifies its bound in seconds and returns a verified 160-truck plan in 3 492 s, 13.3% above a bound that is loose exactly where the crush limit binds; and the comparison with the volumetric benchmark yields a mechanism: the volumetric abstraction is adequate exactly when a

container family’s height-limited and crush-limited layer counts coincide, and credits a truck with roughly twice the placeable count of a family for which they do not. Every plan the method returns is re-verified truck by truck, independently of the solver that produced it. The approach is evaluated on a case study in an automotive seating first-tier supplier against the company’s spreadsheet-based procedure, and against a benchmark model that represents truck capacity volumetrically, whose plans are cheaper on paper to the extent that the truck loads it prescribes cannot be placed in a trailer.

Keywords— Procurement, inventory, transport planning, full-truckload, 3D container loading

1 Introduction

Optimizing the use of resources in manufacturing supply chains is crucial for boosting economic growth, improving social well-being, and minimizing environmental impact. Integrating transport, inventory, and production processes is a fundamental characteristic of supply chains (Stadtler, Kilger, and Meyr 2015) because joint decisions will improve supply chain performance (Díaz-Madroño, Mula, and Peidro 2017).

Procurement refers to the process of acquiring goods and services in a supply chain (Chopra and Meindl 2015). One of the main goals of procurement is to obtain raw materials, parts or finished goods for a factory or distribution centre from a supplier. As mentioned in (Fleischmann, Kopfer, and Sürie 2015), the responsibility for transport planning usually lies with the supplier. However, sometimes the manufacturer makes transport decisions of its supplies in, for example, the automotive industry. Furthermore, an essential asset in the automotive supply chain is not considered in traditional material requirement planning (MRP): returnable containers or returnable transport items (RTI), which are used as tertiary packaging. RTIs enable the storage and transportation of RM and FG through the supply chain. Usually, the manufacturing plant supplies empty RTIs to its supplier, while the supplier delivers full RTIs with the FG or RM. Therefore, during the procurement process, transport and inventory decisions should be integrated, including RTI management. Here, we focus on optimising the operational decisions made during procurement transport planning.

According to (Díaz-Madroño, Peidro, and Mula 2014), using algorithm-based methods for procurement transport planning is more effective than relying on manual approaches based on planners’ intuition. Manual planning can be short-sighted and lead to less efficient outcomes. This becomes even more significant when optimizing procurement transport tasks that aim to reduce the number of trucks and inventory levels, especially under 3D loading constraints where the precise placement of containers within trucks must also be determined. Furthermore, the capacity constraints used in the scientific literature on procurement transport planning are based mainly on freight volume or the total number of loaded product units (Hashemi-Amiri, Ghorbani, and Ji 2023) and (Díaz-Madroño, Peidro, and Mula 2014). The current research seeks to address this gap by introducing a new mixed-integer linear programming (MILP) model. This model uniquely combines procurement transportation with considerations for 3D loading, inventory management, and RTI management. Hereafter, we refer to this model formulation as 3D supply chain procurement transport operational planning (3D-SCPTOP). Additionally, a matheuristic procedure (MATH-3D-SCPTOP) is researched.

In this paper, we present four main contributions: (i) formulating a novel MILP model, 3D-SCPTOP, that integrates procurement transport, RTI management and inventory decisions with stack-based full-truck loading constraints; (ii) reformulating it by Dantzig–Wolfe decomposition over geometrically realisable truck loads, and solving it by a branch and bound over the fleet vector with column generation, a constraint programming pricing oracle and a flow-first construction, giving MATH-3D-SCPTOP; (iii) showing that a volumetric representation of truck capacity, as used in the procurement transport literature, yields plans that are cheaper on paper but cannot be loaded in practice; (iv) validating and testing the proposal with a case study at a first-tier supplier of the automotive sector.

Readers are referred to the typology by (Wäscher, Haußner, and Schumann 2007) for a global view of container loading problems, where the different dimensions of the loading problem are described. Furthermore, (Silva, Toffolo, and Wauters 2019) summarise the exact methods used in the literature to solve container loading problems, and (X. Zhao et al. 2016) explore other solution approaches in the literature. Readers are also referred to the literature reviews of (Mosca, Vidyarthi, and Satir 2019) and (Díaz-Madroño, Peidro, and Mula 2015), which focus more on integrating inventory decisions and transport. Here, we jointly address container loading, procurement transport and inventory decisions.

The paper is organised as follows. Section 2 provides a literature review related to the problem under study. Section 3 presents the problem description. Section 4 describes the 3D-SCPTOP MILP formulation. Section 5 develops the branch and price solution approach, MATH-3D-SCPTOP, together with its pricing

subproblem and bounding arguments. Section 6 reports the computational experiments and Section 7 the case study. Finally, Section 8 concludes and puts forward further research lines.

2 Literature Review

The goal of this research is to integrate procurement transport operational planning decisions with 3D loading restrictions and RTI packaging. Based on the authors' research, no previous work has been identified that meets this criterion. Therefore, three independent, but sometimes overlapping, knowledge bodies have been reviewed: procurement transport and inventory planning, container loading problems and RTI literature.

2.1 Procurement transport and inventory planning

Models that integrate transport and inventory costs during the procurement process are widely studied in the literature. As noted in (Langley et al. 2020), transportation facilitates the movement of raw materials and finished products between suppliers, warehouses, manufacturers, and retailers. Therefore, transport processes play a crucial role in supply chains. (Fleischmann, Kopfer, and Sürie 2015) distinguish between two main types of transportation within a supply chain: "procurement transport," which involves moving materials from suppliers to manufacturing sites, and the "distribution of goods," which concerns delivering finished products from factories to customers. This article focuses specifically on addressing the challenges related to procurement transportation.

There are two main dimensions of procurement transport problems according to the supply chain configuration and the applied transport method. Regarding supply chain configurations (Mosca, Vidyarthi, and Satir 2019): (i) single source to a single destination (SS); (ii) multiple sources to a single destination (MS); (iii) single source to multiple destinations (SM); (iv) multiple sources to multiple destinations (MM). Source components determine the number of supplier locations that are taken into account, while destination components denote the number of supplied manufacturers. Additionally, different transport methods are identified in (Díaz-Madroño, Peidro, and Mula 2014): milk run, less-than-truckload (LTL), full-truckload (FTL). We also analyze the formulation of the transportation capacity constraints in the scientific literature.

(Berman and Wang 2006) address the problem of optimising distribution strategies to deliver products from suppliers to plants, whose aim is to minimise transport, pipeline and plant inventory costs by using cross docks. A mixed integer non-linear programming (MINLP) formulation, a Lagrangian relaxation heuristic, a greedy heuristic, and a branch-and-bound algorithm are proposed to model and solve the problem.

(Sancak and Salman 2011) present a MILP model to optimise procurement transport decisions in a SS supply chain configuration using LTL and FTL transport methods. The objective is to minimise transport and inventory holding costs, while fulfilling production plan requirements on a multiperiod planning horizon. The study explores the possibility of delaying LTL shipments to the subsequent time period, and utilising safety stocks whenever necessary, in a stochastic environment by employing data from a bus manufacturer. The findings demonstrate that delaying shipments leads to reduced holding and transport costs with a minimal stockout risk.

(Schöneberg, Koberstein, and Suhl 2010) examine the concept of delivery profiles, which define the delivery frequency at a fixed rate, offering structured planning, minimizing management effort, and improving coordination in logistics. In this context, FTL transport is used when the goods' volume exceeds a vehicle's capacity, making direct shipment possible. LTL is applied when the volume is insufficient to fill a truck, and the goods are first sent to a consolidation center before reaching the final destination. In (Schöneberg, Koberstein, and Suhl 2013), the proposed model accounts for demand uncertainty.

(Díaz-Madroño, Mula, and Peidro 2014) present a fuzzy multi-objective integer linear programming (FMOILP) model that focuses on operational procurement transport planning in an automotive supply chain, which involves a single source and a single destination while utilising FTL. The developed model aims to minimise the number of trucks employed for procurement transport and the inventory level by considering the uncertainties related to transport capacity levels.

(Hein and Almeder 2016) investigate the benefits of integrated procurement transport and production planning for supply and production processes by comparing a sequential approach, where production is firstly determined, followed by procurement transport decisions, plus an integrated approach in which decisions are jointly made. Numerical experiments reveal that the integrated planning approach leads to substantial cost savings.

(Díaz-Madroño, Mula, and Peidro 2017) present a novel MILP model that integrates procurement, production and transport planning decisions within manufacturing systems. This model extends beyond traditional material requirement planning (MRP) systems by incorporating procurement transport decisions

from multiple sources and a single destination into a unified optimization framework. It considers the capacities of both material and production resources, along with procurement transport planning decisions, by employing various shipping modes such as FTL, LTL, and milk-run.

(Chitsaz, Cordeau, and Jans 2019) address integrating production, inventory and procurement transport into a manufacturing context by defining this problem as an assembly routing problem (ARP). It introduces a novel MILP model for the ARP and proposes a three-phase decomposition matheuristic for its solution.

(Baller et al. 2022) examine the optimization of procurement transport in an automotive factory, focusing on first-tier suppliers at an aggregate level across multiple sources (570) to a single destination, using FTL, LTL, and courier express service (CES) as transportation methods. The proposed MILP approach includes a cyclic inventory constraint, truck utilization, and storage limitations.

(Hashemi-Amiri, Ghorbani, and Ji 2023) propose an integrated approach to optimize a perishable food supply chain. The model combines supplier selection, procurement and distribution transport, production scheduling, and vehicle routing to improve supply chain efficiency and resilience. It handles demand uncertainties through a distributionally robust chance-constrained model. The model is reformulated as MILP by utilizing duality and linearization techniques.

2.2 Procurement Transport with RTIs

Some models further integrate the procurement and delivery of RTIs into the procurement transport literature. In this sense, RTIs are procured to the supplier, who loads the items and sends them to customer. We extend the SC definitions previously defined to contemplate the RTIs management strategies.

[pending: The citation opening this paragraph is missing from the source and could not be identified from the bibliography; the paragraph is left in place and must not be typeset until it is supplied.] Model a food closed-loop supply chain network design problem incorporating returnable transport items (RTIs), product perishability, facility location and demand uncertainty. A two-stage non-linear chance-constrained programming model minimizes fixed facility, production, transport, inventory holding and RTI costs while ensuring a target service level. It's linearized via sample average approximation and distribution-free methods to capture forward RTI-based shipments and reverse flows for reuse. To solve large instances, an improved Lagrangian relaxation algorithm decomposes facility-location and flow decisions, outperforming CPLEX on solution quality and speed.

(Soysal et al. 2023) develop a tactical-level, multi-period closed-loop Vendor-Managed Inventory (VMI) system for RTIs in an agro-food SC. The SM SC integrates forward flows of full RTIs from a single vendor to multiple customers and reverse flows of empties back to the vendor for cleaning. All routing and inventory decisions are made centrally by the vendor under full information sharing. Sustainable plans in terms of amounts of energy spent and empty crates collected are devised and evaluated.

(Guo et al. 2024) model two-tier Container Management Centre (CMC) network. Empty RTIs are dispatched from central CMC hubs to upstream suppliers, loaded with products, carried by truck to downstream customers, then returned empty to regional CMCs for cleaning and storage before being shipped back to central hubs. A neural network is used for demand forecasting. A MILP is proposed that maximises total profit by optimally determining transport costs and the penalties applied to late RTIs returns.

(Y. Zhang et al. 2024) formulate a perishable-food inventory-routing MILP with multiple RTI types in a SM supply chain, integrating procurement of products and pickup of both filled and empty RTIs. Decisions are made on a operational level, optimally determining per period the quantities to pack, deliver, pick up, purchase and lose, as well as the vehicle routes in order to maximise the total profit of the SC that is equal to the total quality-dependent revenue minus the total costs involved, including inventory holding costs, product cost, transportation costs, and RTI purchasing and loss costs. The model is solved exactly via CPLEX solver and approximately via a tailored Kernel Search matheuristic.

2.3 Container loading

Container loading problems encompass a subset of geometric assignment problems known as cutting and packing (*C&P*) problems. They revolve around the allocation of n -dimensional orthotope items in larger n -dimensional orthotope containers. Here the aim is to optimise an objective function while taking into account two key geometric constraints: complete containment of items inside containers and lack of item overlap. A loading pattern serves as a formal representation of a solution to such an assignment problem. Dyckhoff introduced in (Dyckhoff 1990) a typology that unifies *C&P* problems using four different criteria: dimensionality of space, the objective of assignment, containers types and box type. Consequently, (Wäscher, Haußner, and Schumann 2007) develop a new typology that has become the standard in the *C&P* literature. We briefly describe the typology used in (Wäscher, Haußner, and Schumann 2007) based

on five different criteria: (i) dimensionality of space; (ii) kind of assignment (input minimisation of containers for all small items and loading patterns; output maximisation to pack a subset of small items with limited container availability); (iii) assortment of small item by distinguishing if they are either a weakly (or strongly) heterogeneous assortment or identical small items ; (iv) assortment of large objects (if it is either several large objects or one large object); (v) shape of small items (circular or of another shape). Table 1 links the standard *C&P* nomenclature and the criteria of the typology according to (Wäscher, Haußner, and Schumann 2007).

The first mathematical programming model in a scientific journal to describe the container-loading problem was developed in (C. S. Chen, Lee, and Shen 1995), where the authors introduced an MILP model to address the 3D input minimisation problem by considering rotations and containers of varying dimensions. However, it does not specify the use of weakly or strongly heterogeneous boxes or containers. Additionally, real-world constraints are modelled like, for example, weight distribution.

(Junqueira, Morabito, and Sato Yamashita 2012b) propose an alternative MILP formulation to (C. S. Chen, Lee, and Shen 1995) using binary encoding for box position, which utilises normal patterns to reduce possible box coordinates by considering multiples of boxes dimensions and ensuring optimal solutions. Additionally, loading and cargo constraints are developed to ensure the static and dynamic stability of the cargo. Boxes must be supported by other box top faces or the container floor. The same authors integrate further loading restrictions in (Junqueira, Morabito, and Sato Yamashita 2012a), where they integrate multidrop constraints, which address the challenge of packing a truck with boxes for different customers at various destinations. The goal is to plan the container’s loading to minimise the time spent unloading and reloading boxes at each destination.

(W. Zhu and Lim 2012) devise a tree search-based method to solve SLOPP and SKP with 3D constraints. Their approach involves evaluating free spaces by employing a look-ahead tree search and selecting optimal placements based on the best free space-block pairs.

(Gonçalves and Resende 2013) develop a biased random-key genetic algorithm (BRKGA) designed to solve SBSBPP. The algorithm addresses geometric constraints and introduces a maximal-space representation to handle available free spaces in bins. A novel placement procedure is hybridised with the BRKGA, which optimises the order of box packing and the parameters utilised by the placement procedure to offer an effective solution approach to an NP hard problem.

(Bortfeldt and Wäscher 2013) identify constraints for *C&P*, including item-related constraints (orientation constraints, load bearing, loading priorities), container-related constraints (weight distribution and weight limit), and cargo-related constraints (complete-shipment constraints) and load-related constraints (complexity constraints or stability constraints). The study also highlights the achievements and deficits in container loading research that pertain to these constraints.

(Paquay, Schyns, and Limbourg 2016) propose an MILP model for the MBSBPP. One noteworthy constraint is the assumption that containers are truncated parallelepiped instead of rectangular boxes. They also consider constraints, such as box rotation, container load bearing, weight distribution, stability and fragility of items.

(Mahvash, Awasthi, and Chauhan 2018) introduce a novel resolution procedure for the SBSBPP through the column generation technique, where the columns that are generated are the loading configurations of a single bin. For the pricing develop problem they develop a heuristic to find new columns with a negative reduced cost. This approach not only produces better results than the previous articles on the literature, but also gives new lower bounds.

(Silva, Toffolo, and Wauters 2019) study SSLOP models with 3D constraints. These authors compare the different approaches that exist in the literature for model packing problems based on two fundamental approaches: discretised formulation (Junqueira, Oliveira, et al. 2013) and continuous formulation (C. S. Chen, Lee, and Shen 1995) with subsequent modifications. They also indicate that discretised formulation (Junqueira, Morabito, and Sato Yamashita 2012a) performs the best for instances with similar sized boxes. However, it encounters memory limitations for large instances. For other sets of instances, the adapted formulations of (C. S. Chen, Lee, and Shen 1995) and (Paquay, Schyns, and Limbourg 2016) outperform other adaptations. Further research topics suggest including the design of new formulations by considering the limited approaches in the literature.

(Kurpel et al. 2020) introduce techniques and exact approaches to solve input minimisation and output maximisation problems for container-loading problems. Their methods consider practical constraints, such as orientation, separation of boxes and vertical stability. A set partitioning method is also employed in order to obtain upper bounds in a container-loading input minimisation problems. The authors adapt and evaluate four discretisation methods found in the existing literature and incorporate mathematical formulations to address the aforementioned constraints.

(Alonso, Alvarez-Valdes, and Parreño 2020) develop an algorithm by the greedy randomised adaptive

search procedure (GRASP) to tackle a practical container-loading problem with time-sensitive constraints. Their approach addresses dynamic stability constraints while stacking products on pallets and loading them onto trucks during different time periods. Their approach provides high-quality solutions and surpasses an integer programming formulation in efficiency and effectiveness terms.

(Ali et al. 2022) conduct a comprehensive literature review by identifying constraints and solution methods for offline problems with full knowledge of the items to be packed and those tackling the online problems, where items are packed immediately when they arrive, and no prior knowledge of an item is known. The study also identifies and classifies solution approaches for offline packing into six categories: exact methods, constructive heuristics; local search and metaheuristic methods; approximation algorithms; machine learning and hyper-heuristic methods; tree search methods. (Harrath 2022) proposed a three stage heuristic based on layer generation to find good quality solutions the SBSPP problem for large instance up to 993 bins and 200000 items taking into account load balancing constraints.

(Tresca et al. 2022) develop a matheuristic to solve the SBSBPP that includes items with different IDs and different categories, to be delivered to different clients. The algorithm creates monoitem, monocategory, and mixed layers which then are used to compute the final loading configurations of the bins.

(Li, M. Chen, and Huo 2022) develop a hybrid adaptive large neighborhood search algorithm to solve the multiple container loading problem, which tries to pack a given set of items with the minimum number of containers while maximizing the space utilization of them. Therefore, it is both a MBSBPP and a MSSCSP. Their approach takes into account some practical applications such as suspension constraints, rotation and weight limit.

(M. Chen, Huo, and Duan 2023) present a new approach to the 3D-BPP, introducing the 3D-BSDPP which focuses on optimizing both the design of bin sizes and the packing process. A novel Hybrid Biogeography-Based Optimization (HBBO) algorithm is proposed, integrating differential evolution and a local search strategy for enhanced performance.

One noteworthy trend in *C&P* is integrating loading problems with routing. This is known as capacitated vehicle-routing problem with 3D or 2D loading constraints (3L-CVRP or 2L-CVRP). (Junqueira, Morabito, and Sato Yamashita 2012b) present a MILP model with binary encodings for 3L-CVRP by including loading, cargo and multidrop constraints. The combination of these two NP-hard problems requires using heuristics and metaheuristics for real-world instances, as seen in (Dominguez et al. 2016), (Alonso, Martinez-Sykora, et al. 2022) and (Wei et al. 2015). Another noteworthy trend is to use reinforcement learning techniques to solve *C&P*, as in (Que, Yang, and D. Zhang 2023) and (H. Zhao, C. Zhu, and Huang 2022).

2.4 Research Gaps & Methodology

Here, we discuss the main differences between the 3D-SCPTOP model proposal and those reviewed by specifically focusing on procurement transport. The key distinction of the 3D-SCPTOP model compared to the existing literature is the incorporation of 3D constraints and vertical stability constraints to model transport capacity with a view to providing more realistic solutions and reducing the likelihood of infeasible outcomes. Table 1 presents a comparison of inventory modelling, transport modelling, and supply chain configuration between the 3D-SCPTOP model proposal and those currently in the literature. Hence, the 3D-SCPTOP approach is based on a SS supply chain configuration, and takes into account the FTL transportation mode.

Article	SC Configuration	Decision level	Transportation Mode	Transportation Capacity Constraint	Transportation Cost	Inventory Capacity Restriction	RTIs	Inventory Costs	Safety Stocks	Modelling Framework
(Guo et al. 2024)	MM	TO	FTL	UH	X	X	S-NF	X		MINLP
(Berman and Wang 2006)	MM	T	FTL-LTL	V	X			X		MINLP
(Sancak and Salman 2011)		O	LTL-FTL	V	X			X	X	MILP
(Schöneberg, Koberstein, and Suhl 2010)	MS	T	FTL-LTL	V	X	X		X		MILP
(Schöneberg, Koberstein, and Suhl 2013)	MS	O	FTL-LTL	V	X	X		X		MILP
(Díaz-Madroño, Mula, and Peidro 2014)	SS	O	FTL	UH	X			X	X	FMOILP
(Hein and Almeder 2016)	MS	O	MR	V	X			X		MILP
(Díaz-Madroño, Mula, and Peidro 2017)	MS	T	FTL-LTL-MR	V	X	X		X	X	MILP
(Meyer and Amberg 2018)	MS	T-O	MR-FTL	V	X	X		X		MILP
(Chitsaz, Cordeau, and Jans 2019)	MS	O-T	MR	V	X	X		X		MILP
(Baller et al. 2022)	MS	O	MR-FTL-LTL-CES	V-W	X	X			X	MILP
(Hashemi-Amiri, Ghorbani, and Ji 2023)	MS	T	MR	V	X					CCP
3D-SCPTOP	SS	O	FTL	3D-Constraints	X	X		X	X	MILP
(Y. Zhang et al. 2024)	MS	TO	MR	UH	X	X	M-NF	X		ILP

Abbreviations. **T** (tactical), **O** Operational; **FTL** (full-truck load), **LTL** (less-than-truck load), **MR** (milk-run), **CES** (courier express service); **V** (volumetric), **W** (weight), **UH** Unit Holding; **CCP** chance-constrained programming

Table 1: Comparison of procurement transport models

On the other hand, the 3D-SCTOP model diverges from those proposed in the packaging literature in several aspects. Firstly, the loading problem takes into account different time periods. Therefore, the

Article	Problem type	Resolution procedure	Orientation	Weight	Load Balance	Stability	Time-Dependent
(W. Zhu and Lim 2012)	SLOP-SKP	TSM	X	X			
(Gonçalves and Resende 2013)	SBSBPP	BRKGA	X	X		X	
(Paquay, Schyns, and Limbourg 2016)	SBSBPP	EM	X	X	X	X	
(Mahvash, Awasthi, and Chauhan 2018)	SBSBPP	CG	X	X			
(Alonso, Alvarez-Valdes, and Parreño 2020)	SSSCSP	GRASP	X	X	X	X	X
(Tresca et al. 2022)	SBSBPP	MH	X	X	X	X	
(Harrath 2022)	SBSBPP	H	X	X	X		
(Li, M. Chen, and Huo 2022)	SBSBPP-MSSCSP	HALNs	X	X		X	
(M. Chen, Huo, and Duan 2023)	BSDPP	HBBO	X	X			
3D-SCPTOP	SSSCSP & MILOPP	MH	X	X	X	X	X

Abbreviations. **TSM** (Tree search method), **BRKGA** Biased Randomized Jey Genetic Algorithm; **EM** (Exact Methods), **MH** (MatHeuristic), **CG** (Column Generation), **H** (Heuristic); **HCLP** (Hybrid Adaptive Large Neighborhood Search), **HBBO** (Hybrid Biogeography-Based Optimization)

Table 2: Comparison of C&P problems

specific container (small items or boxes in the *C&P* literature) that we utilise are yet to be defined, as the choice of container depends on the inventory levels of each product. Also, the objective function depends on inventory and transport costs, which are inherently connected to the loading problem at hand. Therefore, we simultaneously solve an input minimisation and output maximisation problem. Both of these are a result of transport minimisation and inventory costs. Input minimisation comes because we want to find the minimum number of trucks needed. Meanwhile, output maximization comes from the FTL and the transport costs.

Based on the literature review and the identified research gaps, our proposal is structured according to the steps depicted in Figure 1.

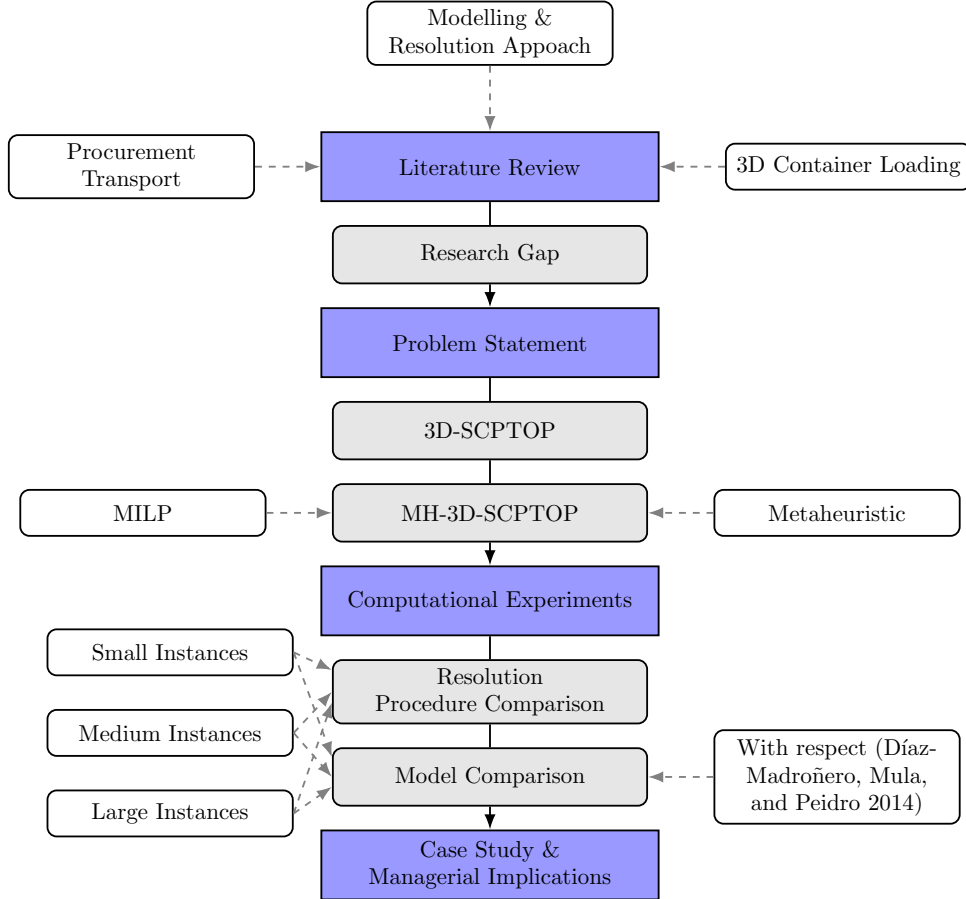


Figure 1: Research Methodology

3 Problem Description

The objective is to minimise the transportation and inventory holding costs of a manufacturing site by integrating procurement transport, RTI management and inventory decisions with full-truckload (FTL) loading constraints expressed in a stack-based form. The supply chain under study has a single source and a single destination. Products are shipped in different RTI types. RTIs are not placed individually in three dimensions but are grouped into vertical *stacks* whose 2D footprints are placed on the truck floor. Each stack type has a fixed footprint and is compatible with a subset of RTI types; each layer of a stack is homogeneous (all RTIs in a layer share a single type), while different layers of the same stack may carry different compatible RTI types. The total stack height is bounded by the truck height and the weight borne by the bottom of each stack is bounded by a crush-limit cap. Trucks carry full RTIs from the supplier to the plant and return folded empties from the plant to the supplier; folding affects RTI height, and hence the number of layers that fit in a stack, but not the stack footprint.

The assembler’s demand drives the procurement process because it determines the optimal quantity of each product to procure from the supplier to the manufacturing site. Raw materials and assembled parts are produced at the supplier, packed in their corresponding RTI, and transported under FTL. Each RTI carries a single product type at any given time, and each product is shipped in a single RTI type. Once full RTIs arrive at the plant, the products are consumed and the RTIs are emptied, folded and returned to the supplier.

Stacks must not overlap and must be contained within the truck’s floor; the truck’s total weight is bounded; and the per-stack weight cap is respected with *period-exact* knowledge of the products loaded in each slot, so that crush-limit tests reflect the actual period’s load rather than a worst-case bound. Stacks may be rotated by 90° on the truck floor. Because stacks are self-supporting by construction, no additional vertical-stability constraints are required. As FTL is the mode of transport, a dispatched truck must be full — but a truck may be full either because it has run out of space or because it has run out of payload, so the requirement is imposed on whichever of the two resources binds. Minimum coverage stock is enforced on future demand.

Initial data.

- **Product data:** initial inventory, periodic demand (in full RTIs) at the plant and at the supplier, product-to-RTI mapping, weight of each full RTI.
- **RTI data:** height when full and when folded, weight of a folded RTI, volume.
- **Stack data:** footprint (p_h, q_h) of each stack type $h \in H$, set of compatible RTI types $R(h)$, and, for each $r \in R(h)$, number of RTIs per layer n_{hr} .
- **Transport data:** number of available trucks, truck length/width/height, truck weight capacity, per-stack weight cap, minimum truck utilisation on the binding resource.

To determine.

- A set of loading configurations (LCs), each specifying the stack-type assignment, 2D placement and orientation of its slots, and the number of layers of each compatible RTI type in each slot.
- The number of trucks used per day and the LC they employ.
- The number of full RTIs of each product loaded in each slot per period (plant-bound trucks) and the number of folded RTIs of each type loaded per truck (supplier-bound trucks).
- The resulting inventory of full and folded RTIs at both nodes.

Subject to.

- 2D non-overlap and containment of stacks on the truck floor, with allowed 90° rotation.
- Per-stack height budget.
- Per-stack crush-limit weight cap, period-exact on plant-bound trucks and deterministic on supplier-bound trucks.
- Per-truck weight capacity.
- Minimum truck utilisation, on volume or on payload, whichever binds.
- Minimum coverage stock.

In order to:

- Minimise total transportation cost.
- Minimise total inventory holding cost.

4 Mathematical Model

The 3D-SCPTOP formulation integrates stack-based truck-loading constraints with procurement transport and inventory decisions. It departs from a standard 3D container-loading formulation in two related ways. First, the 3D placement of individual RTIs inside the truck is replaced by a 2D placement of variable-composition *stacks* on the truck floor, with per-stack height and weight caps. Second, the per-period decision of which product fills each slot is retained, but the crush-limit cap is evaluated slot-by-slot using the period's actual product mix. This yields a tighter per-stack weight bound than a worst-case-per-RTI-type abstraction, while eliminating vertical-stability constraints altogether, since stacks are self-supporting by construction.

Notational conventions. Throughout this section, $g \in G = \{s, p\}$ indexes nodes (supplier and plant), $n \in N$ products, $r \in R$ RTI types, $h \in H$ stack types, $i, j \in I$ stack-slot positions within a loading configuration, $l \in L$ loading configurations, $k \in K$ trucks and $t \in T$ planning periods. Variables whose direction is determined by definition omit the g subscript: V_{klnt} and $V_{wslot_{kilnt}}$ denote full-RTI loads on plant-bound trucks, and V_{klrt}^e denotes folded-RTI loads on supplier-bound trucks. Table 3 collects the complete nomenclature.

Table 3: Nomenclature of 3D-SCPTOP (stack-based reformulation)

Indices and sets	
G	Supplier and plant nodes, $G = \{s, p\}$.
N	Products, $n = 1, \dots, N$.
R	RTI types, $r = 1, \dots, R$.
H	Stack types, $h = 1, \dots, H$.
I	Stack-slot indices within a loading configuration, $i = 1, \dots, I $ (see eq. 4.1).
L	Loading configurations, $l = 1, \dots, L$.
K	Trucks, $k = 1, \dots, K$.
T	Planning periods, $t = 1, \dots, T$.
$M(r)$	Products carried by RTI type r , $M(r) \subseteq N$.
$R(h)$	RTI types compatible with stack type h , $R(h) \subseteq R$.
Parameters	
cK	Cost of using a truck.
lt	Lead transport time between plant and supplier (and vice versa).
ig_n	Initial inventory of $n \in N$ at $g \in G$ (full RTIs).
ie_{gr}	Initial inventory of folded RTIs $r \in R$ at $g \in G$.
ic_{gn}	Inventory cost of $n \in N$ at $g \in G$.
d_{gnt}	Production plan for $n \in N$ at $g \in G$ in $t \in T$.
q_{nt}	Initial arrivals of full RTIs of product n at p in $t \in \{1, \dots, lt\}$.
q_{rt}	Initial arrivals of folded RTIs of type r at s in $t \in \{1, \dots, lt\}$.
ss_{gnt}	Security stock of n at g in t (full RTIs).
ss_{grt}	Security stock of folded RTIs r at g in t .
w_n	Weight of a full RTI carrying product n .
w_r^f	Weight of a folded RTI of type r .
pK^w	Maximum truck weight capacity.
$W^{\max}$	Per-stack crush-limit weight cap.
pK, qK, rK	Truck length, width, and height.
Γ_K	Truck volume, $\Gamma_K = pK \cdot qK \cdot rK$.
α	Minimum truck utilisation, required on whichever of volume or payload binds ($0 < \alpha \leq 1$).
p_h, q_h	Footprint length and width of stack type h .
r_{gr}	Height of one RTI of type r at node g (full at p , folded at s).
γ_{gr}	Volume of one RTI of type r at node g .

n_{hr}	RTIs of type $r \in R(h)$ per layer of stack type h .
M	A sufficiently large constant.
Decision variables – LC level	
Z_{gl}	$\in \{0, 1\}$. Equals 1 iff LC l is active in direction g .
θ_{gl}	$\in \{0, 1\}$. Selects which resource the utilisation floor is applied to: volume if 1, payload if 0.
X_{gihl}	$\in \{0, 1\}$. Equals 1 iff slot i of LC l is of stack type h in direction g .
L_{gihlr}	$\in \mathbb{Z}_{\geq 0}$. Number of layers of RTI type $r \in R(h)$ in slot i of LC l in direction g .
LX_{gihl}	$\in \{0, 1\}$. Equals 1 iff stack h 's length is along the x -axis in slot i .
LY_{gihl}	$\in \{0, 1\}$. Equals 1 iff stack h 's length is along the y -axis in slot i .
PX_{gil}, PY_{gil}	≥ 0 . Bottom-left coordinates of slot i in LC l in direction g .
A_{gijl}	$\in \{0, 1\}$. Equals 1 iff slot i is to the left of slot j in LC l .
B_{gijl}	$\in \{0, 1\}$. Equals 1 iff slot i is to the right of slot j in LC l .
C_{gijl}	$\in \{0, 1\}$. Equals 1 iff slot i is behind slot j in LC l .
D_{gijl}	$\in \{0, 1\}$. Equals 1 iff slot i is in front of slot j in LC l .
Decision variables – runtime level	
Y_{gkt}	$\in \{0, 1\}$. Equals 1 iff truck k is used in direction g in period t .
Y_{gklt}	$\in \{0, 1\}$. Equals 1 iff truck k uses LC l in direction g in period t .
$V_{kln t}$	$\in \mathbb{Z}_{\geq 0}$. Full RTIs of product n on plant-bound truck k using LC l in period t .
V_{kilnt}^{slot}	$\in \mathbb{Z}_{\geq 0}$. Full RTIs of product n in slot i of plant-bound truck k using LC l in period t .
V_{klrt}^e	$\in \mathbb{Z}_{\geq 0}$. Folded RTIs of type r on supplier-bound truck k using LC l in period t .
I_{gnt}	$\in \mathbb{Z}_{\geq 0}$. End-of-period inventory of n at g in t (full RTIs).
$I_{e_{gtr}}$	$\in \mathbb{Z}_{\geq 0}$. End-of-period inventory of folded RTIs r at g in t .
Q_{nt}	$\in \mathbb{Z}_{\geq 0}$. Full RTIs of product n in transit to the plant in t .
Q_{rt}	$\in \mathbb{Z}_{\geq 0}$. Folded RTIs of type r in transit to the supplier in t .

Objective

$$\text{Min } z = \sum_{g \in G} \sum_{k \in K} \sum_{t \in T} cK \cdot Y_{gkt} + \sum_{g \in G} \sum_{n \in N} \sum_{t \in T} ic_{gn} \cdot I_{gnt} \quad (1)$$

Inventory and flow balance

The inventory balance block is structurally identical to the original formulation. For completeness it is reproduced below.

$$I_{pnt} = I_{p,n,t-lt} - d_{pnt} + Q_{n,t-lt} \quad \forall n \in N, t \geq lt + 1 \quad (2)$$

$$I_{pnt} = I_{p,n,t-1} - d_{pnt} + q_{nt} \quad \forall n \in N, t \in \{2, \dots, lt\} \quad (3)$$

$$I_{pnt} = i_{pn} - d_{pnt} + q_{nt} \quad \forall n \in N, t = 1 \quad (4)$$

$$I_{e_{ptr}} = I_{e,p,t-1,r} + \sum_{n \in M(r)} d_{pnt} - Q_{rt} \quad \forall r \in R, t \geq 2 \quad (5)$$

$$I_{e_{ptr}} = i_{e_{pr}} + \sum_{n \in M(r)} d_{pnt} - Q_{rt} \quad \forall r \in R, t = 1 \quad (6)$$

$$I_{snt} = I_{s,n,t-1} + d_{snt} - Q_{nt} \quad \forall n \in N, t \geq 2 \quad (7)$$

$$I_{snt} = i_{sn} + d_{snt} - Q_{nt} \quad \forall n \in N, t = 1 \quad (8)$$

$$I_{e_{str}} = I_{e,s,t-lt,r} + Q_{r,t-lt} - \sum_{n \in M(r)} d_{snt} \quad \forall r \in R, t \geq lt + 1 \quad (9)$$

$$I_{e_{str}} = I_{e,s,t-1,r} + q_{rt} - \sum_{n \in M(r)} d_{snt} \quad \forall r \in R, t \in \{2, \dots, lt\} \quad (10)$$

$$I_{e_{str}} = i_{e_{sr}} + q_{rt} - \sum_{n \in M(r)} d_{snt} \quad \forall r \in R, t = 1 \quad (11)$$

$$I_{gnt} \geq ss_{gnt}, \quad I_{e_{gtr}} \geq ss_{gtr} \quad \forall g \in G, n \in N, r \in R, t \in T \quad (12)$$

Stack packing constraints

Exactly one stack type can be assigned to a slot, and only if its LC is active:

$$\sum_{h \in H} X_{ghl} \leq Z_{gl} \quad \forall g \in G, i \in I, l \in L. \quad (13)$$

The stack occupying slot i is oriented either with its length along the x -axis or along the y -axis:

$$LX_{ghl} + LY_{ghl} = X_{ghl} \quad \forall g \in G, h \in H, i \in I, l \in L. \quad (14)$$

The total stack height cannot exceed the truck height. The following constraint also forces $L_{ghlr} = 0$ whenever $X_{ghl} = 0$, since $r_{gr} > 0$:

$$\sum_{r \in R(h)} r_{gr} L_{ghlr} \leq rK \cdot X_{ghl} \quad \forall g \in G, h \in H, i \in I, l \in L. \quad (15)$$

Slots must not overlap. For each pair $i \neq j$ in LC l :

$$PX_{gil} + \sum_{h \in H} (p_h LX_{ghl} + q_h LY_{ghl}) \leq PX_{gjl} + M(1 - A_{gil}) \quad (16)$$

$$PX_{gjl} + \sum_{h \in H} (p_h LX_{ghl} + q_h LY_{ghl}) \leq PX_{gil} + M(1 - B_{gjl}) \quad (17)$$

$$PY_{gil} + \sum_{h \in H} (q_h LX_{ghl} + p_h LY_{ghl}) \leq PY_{gjl} + M(1 - C_{gil}) \quad (18)$$

$$PY_{gjl} + \sum_{h \in H} (q_h LX_{ghl} + p_h LY_{ghl}) \leq PY_{gil} + M(1 - D_{gjl}) \quad (19)$$

with the cover constraint ensuring that whenever both slots are used at least one relative-position variable is active:

$$A_{gil} + B_{gjl} + C_{gil} + D_{gjl} \geq \sum_h X_{ghl} + \sum_h X_{ghl} - 1 \quad \forall g, l, i \neq j. \quad (20)$$

Containment in the truck floor:

$$PX_{gil} + \sum_{h \in H} (p_h LX_{ghl} + q_h LY_{ghl}) \leq pK + M(1 - \sum_h X_{ghl}) \quad \forall g, i, l, \quad (21)$$

$$PY_{gil} + \sum_{h \in H} (q_h LX_{ghl} + p_h LY_{ghl}) \leq qK + M(1 - \sum_h X_{ghl}) \quad \forall g, i, l. \quad (22)$$

Minimum truck utilisation. Full-truckload transport requires a dispatched truck to be full, but a truck can be full in two different ways: it may run out of space or it may run out of payload. Imposing the requirement on volume alone is not a conservative simplification, it is an infeasible one — filling a fraction α of a 97.9 m³ trailer with RTIs at 590 kg/m³ weighs 27.4 t against a 24 t cap, so no truck is dispatchable at all and the instance is empty. Summing the two ratios instead is the opposite error: it accepts a truck that is 35% full by volume and 20% by weight, which is full by neither. We therefore require a truck to reach a fraction α of *either* resource, using a binary θ_{gl} to select which one binds:

$$\sum_{i \in I} \sum_{h \in H} \sum_{r \in R(h)} \gamma_{gr} n_{hr} L_{ghlr} \geq \alpha \Gamma_K (Z_{gl} - 1 + \theta_{gl}) \quad (23)$$

$$\sum_{i \in I} \sum_{h \in H} \sum_{r \in R(h)} \omega_{gr} n_{hr} L_{ghlr} \geq \alpha pK^w (Z_{gl} - \theta_{gl}) \quad \forall g \in G, l \in L, \quad (24)$$

with $\omega_{gr} = w_r^f$ in the supplier direction and $\omega_{pr} = \min_{n \in M(r)} w_n$ in the plant direction, the lightest product being the one that makes the requirement hardest to meet. The same disjunction is imposed on each dispatched truck at runtime, on the loads V_{klnt} and V_{klrt}^e rather than on the configuration's designed capacity, since a truck may be loaded to less than the configuration allows.

Truck-LC linking and runtime loading

A truck uses at most one LC in a period, and can only use an active LC:

$$\sum_{l \in L} Y_{gklt} = Y_{gkt} \quad \forall g, k, t \quad (25)$$

$$Y_{gklt} \leq Z_{gl} \quad \forall g, k, l, t \quad (26)$$

For plant-bound trucks, full RTIs of type r loaded in slot i cannot exceed the layer capacity of the stack placed in that slot; products sharing the same RTI type partition that slot-capacity:

$$\sum_{n \in M(r)} V_{kilnt}^{\text{slot}} \leq \sum_{h: r \in R(h)} n_{hr} L_{p,i,h,l,r} + M(1 - Y_{pkt}) \quad \forall k, i, l, r, t. \quad (27)$$

Truck-level aggregation of per-slot loads:

$$V_{klnt} = \sum_{i \in I} V_{kilnt}^{\text{slot}} \quad \forall k, l, n, t. \quad (28)$$

For supplier-bound trucks, folded RTIs of type r are capped by the aggregate layer capacity of the LC in use (no slot-level tracking is needed because folded-RTI weight is deterministic in r):

$$V_{klrt}^e \leq \sum_{i \in I} \sum_{h: r \in R(h)} n_{hr} L_{s,i,h,l,r} + M(1 - Y_{skt}) \quad \forall k, l, r, t. \quad (29)$$

Weight constraints

Per-truck weight, plant-bound:

$$\sum_{l \in L} \sum_{n \in N} w_n V_{klnt} \leq pK^w Y_{pkt} \quad \forall k, t. \quad (30)$$

Per-truck weight, supplier-bound:

$$\sum_{l \in L} \sum_{r \in R} w_r^f V_{klrt}^e \leq pK^w Y_{skt} \quad \forall k, t. \quad (31)$$

Per-stack crush-limit, period-exact on plant-bound trucks (the cap reflects the actual product mix loaded in each slot):

$$\sum_{n \in N} w_n V_{kilnt}^{\text{slot}} \leq W^{\max} \quad \forall k, i, l, t. \quad (32)$$

Per-stack crush-limit on supplier-bound trucks (deterministic in the LC since folded-RTI weight depends only on r):

$$\sum_{h \in H} \sum_{r \in R(h)} w_r^f n_{hr} L_{s,i,h,l,r} \leq W^{\max} \quad \forall i, l. \quad (33)$$

Flow aggregation

$$Q_{nt} = \sum_{k \in K} \sum_{l \in L} V_{klnt} \quad \forall n, t \quad (34)$$

$$Q_{rt} = \sum_{k \in K} \sum_{l \in L} V_{klrt}^e \quad \forall r, t \quad (35)$$

4.1 Symmetry breaking and slot-set sizing

The slots $i \in I$ of a given LC are interchangeable, so any solution admits $|I|!$ permutations of the slot index. We break this symmetry by enforcing that slot $i + 1$ is used only if slot i is used:

$$\sum_{h \in H} X_{g,i+1,h,l} \leq \sum_{h \in H} X_{g,i,h,l} \quad \forall g \in G, l \in L, i \in \{1, \dots, |I| - 1\}. \quad (36)$$

The cardinality of I is a modelling parameter. An upper bound that eliminates provably unused slot variables without cutting any feasible layout is

$$|I| \leq \left\lfloor \frac{pK \cdot qK}{\min_{h \in H} p_h \cdot q_h} \right\rfloor, \quad (37)$$

corresponding to tiling the truck floor with the smallest-footprint stack type; it serves as a tight preprocessing limit on $|I|$.

Domains

$$\begin{aligned} Z_{gl}, \theta_{gl}, X_{ghl}, LX_{ghl}, LY_{ghl}, A_{gijl}, B_{gijl}, C_{gijl}, D_{gijl}, Y_{gkt}, Y_{gklt} &\in \{0, 1\}, \\ L_{ghl}, V_{klt}, V_{kilt}^{\text{slot}}, V_{klrt}^e, I_{gnt}, I_{gtr}, Q_{nt}, Q_{rt} &\in \mathbb{Z}_{\geq 0}, \\ PX_{gil}, PY_{gil} &\geq 0. \end{aligned}$$

5 Branch and Price Solution Approach

There are three structural difficulties with the 3D-SCPTOP model of Section 4. The first is that a priori we do not know how many loading configurations are needed, so $|L|$ must be overestimated; every configuration that turns out to be unnecessary still carries its full block of placement variables. The second is that the model replicates a complete two-dimensional packing problem $|G| \cdot |L|$ times, and each replica contributes $4 \binom{|L|}{2}$ big- M disjunctions (16)–(19) whose linear relaxation is weak. The third is that the model indexes its variables by truck, and trucks are interchangeable: any two used trucks of the same direction and period can swap their entire loads, so every solution appears in the search a factorial number of times.

The third difficulty is the one that scales worst, because it does not depend on the loading geometry at all. It is present in the volumetric benchmark of Appendix A exactly as in 3D-SCPTOP, and Section 6 reports what it costs there: on the larger instances the compact model’s search tree grows until it reaches the memory budget it is given, with its dual bound stalled. Operational procurement planning lives at horizons of weeks over dozens of trucks, so that is the regime the method has to serve, and it is the regime in which a decomposition earns its place. A column generation scheme based on branch and price is therefore developed.

The key components of the branch and price algorithm are the master problem (MP), the restricted master problem (RMP), the pricing problem (PP) and the branching policy. The MP is a reformulation of the model that exploits its decomposable structure. Inspecting Section 4, every geometric constraint (13)–(23) carries a single configuration index l , and every loading constraint (27)–(33) carries a single truck index k and a single period t ; the transport and inventory block sees them *only* through the aggregate flows Q_{nt} and Q_{rt} of (34)–(35). The model is therefore block-angular, with one independent block per dispatched truck. This suggests taking as the unit of decision neither the individual RTI nor the loading configuration, but the *complete loaded truck*. A column is then a load, and the master decides how many trucks carry it, so the trucks are never distinguished in the first place and the symmetry disappears by construction.

The MP is typically too large to be solved directly because it involves a huge number of columns representing all possible truck loads. To manage this complexity we consider only a smaller subset of columns, which leads to the RMP. Solving the RMP provides an optimum for the included subset, but not necessarily for the entire column set. RMP integrality constraints are therefore relaxed to obtain dual variables, and the PP uses those duals to identify columns of negative reduced cost that should be added to the RMP. If no such column exists, the integrality-relaxed MP has been solved to optimality. If all the variables associated with the solution are integers, an integer solution to the current branch has been found. If not, the branch must be divided by adding further constraints. Figure 2 exemplifies the algorithm.

5.1 Formulation of the Master Problem and the Restricted Master Problem

The key idea of the set-partitioning formulation for 3D-SCPTOP is to enumerate all the feasible *truck loads* with their associated costs. A plant-bound truck load is a vector $\mathbf{a} = (a_n)_{n \in N}$ of full RTIs for which there exist a slot count $m \leq |I|$, stack types $h_1, \dots, h_m \in H$, orientations, bottom-left coordinates (x_i, y_i) , layer

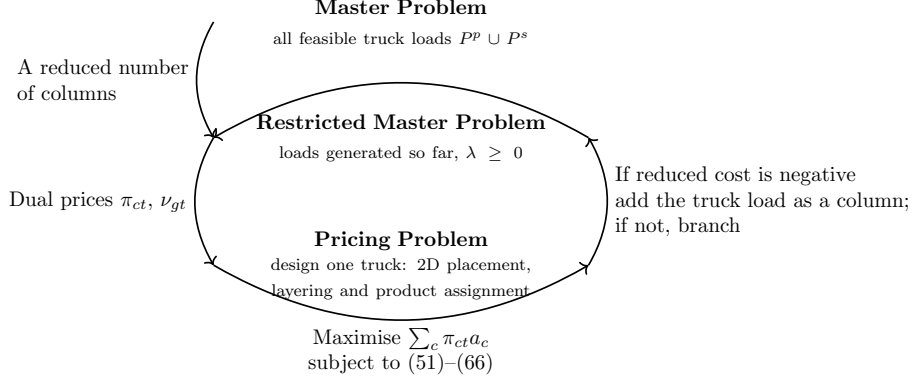


Figure 2: Branch and price scheme for 3D-SCPTOP. Every geometric, height, crush-limit, payload and full-truckload constraint is enforced inside the pricing problem, so each column is a physically loadable truck.

counts λ_{ir} and per-slot product loads v_{in} such that

$$\text{the } m \text{ footprints, rotated as chosen, do not overlap and lie inside } pK \times qK, \quad (38)$$

$$\sum_{r \in R(h_i)} r_{pr} \lambda_{ir} \leq rK \quad \forall i \leq m, \quad (39)$$

$$\sum_{n \in M(r)} v_{in} = n_{h_i r} \lambda_{ir} \quad \forall i \leq m, r \in R(h_i), \quad (40)$$

$$\sum_{n \in N} w_n v_{in} \leq W^{\max} \quad \forall i \leq m, \quad (41)$$

$$\sum_{n \in N} w_n a_n \leq pK^w, \quad \max \left\{ \frac{\sum_n \gamma_{pp(n)} a_n}{\Gamma_K}, \frac{\sum_n w_n a_n}{pK^w} \right\} \geq \alpha, \quad (42)$$

$$a_n = \sum_{i \leq m} v_{in} \quad \forall n \in N. \quad (43)$$

Let P^p be the set of all such loads, and P^s the analogous set of supplier-bound loads $\mathbf{b} = (b_r)_{r \in R}$, built with folded heights r_{sr} , folded weights w_r^f and folded volumes γ_{sr} ; there no product assignment is needed, because the weight of a folded RTI depends only on its type.

Two features of this definition matter. First, a column carries the *per-slot* product assignment v_{in} , so the crush limit (41) is evaluated on the product mix actually riding in that slot. The column therefore reproduces the period-exact cap (32) of the compact model, and not the conservative surrogate $\bar{w}_r = \max_{n \in M(r)} w_n$ that a pool of capacity-only configurations is forced to use. Second, a column is a feasibility certificate: the placement $(h_i, x_i, y_i, \text{rotation})$ witnessing $\mathbf{a} \in P^p$ is retained, so any solution can be handed to a loading crew as a drawing and re-verified independently of the solver that produced it.

Constraint (42) is the same disjunction as (23)–(24), written on the load a truck actually carries rather than on the capacity its configuration provides. Both regimes occur in practice: coarse containers fill the trailer by volume, dense ones by weight.

Let $\lambda_{pt}^g \in \mathbb{Z}_{\geq 0}$ count the trucks dispatched in direction g and period t carrying load p , and let a_n^p and a_r^p be the RTIs of product n , respectively of type r , that load p contains. Every truck costs cK . The MP of our problem is

$$\min \quad cK \sum_{g \in G} \sum_{t \in T} \sum_{p \in P^g} \lambda_{pt}^g + \sum_{g \in G} \sum_{n \in N} \sum_{t \in T} i_{cgn} I_{gnt} \quad (44)$$

subject to

$$\sum_{p \in P^p} a_n^p \lambda_{pt}^p = Q_{nt} \quad \forall n \in N, t \in T \quad (45)$$

$$\sum_{p \in P^s} a_r^p \lambda_{pt}^s = Q_{rt} \quad \forall r \in R, t \in T \quad (46)$$

$$\sum_{p \in P^g} \lambda_{pt}^g \leq |K| \quad \forall g \in G, t \in T \quad (47)$$

(2)–(12) (inventory balances and coverage stock)

$$\lambda_{pt}^g \in \mathbb{Z}_{\geq 0} \quad \forall g \in G, p \in P^g, t \in T.$$

Objective (44) minimises transport plus inventory holding cost. Constraints (45)–(46) state that the flow of full and folded RTIs is exactly what the dispatched trucks carry, and (47) caps the fleet available per direction and period. The coverage-stock rows (12) are carried over for both nodes and both kinds of stock, full and folded, exactly as the compact model states them (Section 5.6).

MP and 3D-SCPTOP have the same optimal value whenever $|L|$ is not restricted a priori. Given a 3D-SCPTOP solution, each used truck (k, t) induces a load vector satisfying (38)–(43), obtained by reading the slot data off its configuration and setting $v_{in} = V_{kilt}^{\text{slot}}$; counting trucks by load vector gives a MP solution of equal cost. Conversely, introducing one configuration per distinct load used by a MP solution and assigning trucks accordingly reconstructs a 3D-SCPTOP solution of the same cost. This equivalence is what distinguishes the present approach from a pool-based method such as PLC-SCPTOP: a predefined pool is a *restriction* and can only yield upper bounds of unknown quality, whereas MP is the same problem written differently, so any bound derived from it is a bound on the original.

The RMP replaces P^g by a subset $\bar{P}^g$ and relaxes integrality,

$$\min \quad cK \sum_{g \in G} \sum_{t \in T} \sum_{p \in \bar{P}^g} \lambda_{pt}^g + \sum_{g \in G} \sum_{n \in N} \sum_{t \in T} i_{cgn} I_{gnt} \quad (48)$$

subject to (45)–(47) restricted to $\bar{P}^g$, with $\lambda_{pt}^g \geq 0$. Dual values are obtained by solving the RMP. Let π_{nt} and π_{rt} be the duals of (45)–(46) and $\nu_{gt} \leq 0$ those of (47). The reduced cost associated with a plant-bound load $p \in P^p$ dispatched in period t is

$$\bar{c}(p, t) = cK - \sum_{n \in N} \pi_{nt} a_n^p - \nu_{pt}, \quad (49)$$

and analogously in the supplier direction. A load with negative reduced cost can improve the current RMP solution. For each iteration we solve the subproblem of finding the load of minimum reduced cost; loads with negative reduced cost are added as new columns, the RMP is re-solved to generate new duals, and the process repeats until no further improving column is discovered.

Three implementation details of the RMP matter for what follows. First, the RMP carries *artificial* variables on the linking rows (45)–(46) and on the coverage-stock rows, penalised at a large cost M in the objective, so that it is feasible for any pool and always yields duals; a solution with positive artificials is not a plan. Second, it carries cumulative cover cuts: by period t the plant must have received cumulative demand plus coverage stock, and dividing that requirement by the most a single truck can carry bounds the number of trucks dispatched up to $t - lt$ from below. They tighten the relaxation considerably, and their duals, like those of every row the search later adds, are folded into ν_{gt} so that (49) remains the exact reduced cost. Third, the RMP is *period-scoped*: a column priced against the duals of period t receives a dispatch variable at t alone, and only the shared initial pool is offered to every period. This changes the size of the restricted master, not the master — the pricing subproblem is still solved per (g, t) over the whole of P^g , so a sweep that finds nothing at any pair still proves that the restricted LP equals the master LP.

5.2 Lower bounds

Care is needed here, because the natural candidate is not a bound at all. The RMP is a *restriction* of the MP: it optimises over $\bar{P}^g \subseteq P^g$, so its LP value satisfies $z_{\text{RMP}} \geq z_{\text{MP}}$. Until column generation has converged the RMP value is an *upper* bound on the master LP, and reporting it as a lower bound is simply wrong. Three rules decide what may be reported; Figure 3 states them as a decision.

A certificate needs a full sweep. When the PP has *proven* that $\zeta_{gt} := \max_{p \in P^g} \sum_c \pi_{ct} a_c^p \leq cK - \nu_{gt}$ for every g and t , no column outside $\bar{P}^g$ has negative reduced cost, so the current duals are feasible for the dual of the full MP. Weak duality then gives $z_{\text{RMP}} = z_{\text{MP}} \leq z_{\text{3D-SCPTOP}}^*$. The proof is only as good as the exhaustiveness of the sweep behind it. Two economies that are legitimate while columns are still coming — pricing only the pairs whose aggregates are fractional, and skipping a pair whose duals round to something already priced — forfeit the right to certify, because a pair nobody looked at proves nothing. Certification therefore runs one sweep over *every* (g, t) , with no memoisation and no restriction to the pairs a branch has touched, and every call must come back *proved*: an oracle stopped on its time limit reports itself unproven, and a sweep with one such pair proves nothing about the whole. When any of this fails the node carries its parent’s bound, which remains valid for the child because the child’s region is a subset of the parent’s.

The bound is the raw LP value. The RMP objective includes the penalty M on the artificial variables. The cost of a plan is the value *net* of that penalty, and that is the right number to report as a cost; the bound is the *raw* value with the penalty included, because the penalty is part of the LP whose dual the certificate rests on. Positive artificials at convergence mean the region cannot be made feasible by any column of P^g , so the region is infeasible in the true problem and is pruned rather than bounded.

The restricted value is not a bound. At every node the search first solves the LP over the current pool. That value sits *above* the region’s true LP value and is used only as a filter: if even the over-estimate lies below the incumbent the region cannot be pruned and the expensive sweep is not run. It is never entered as a bound, and a search stopped by its deadline reports the last *certified* value it holds, not the last LP it solved. Reduced costs oscillate here, and a sweep that finds nothing is not convergence unless every pair was proved.

A bound by inclusion. The certificate above is expensive when the oracle is the constraint programme of Section 5.3; the following observation makes a cheaper one available for the geometric problem.

Proposition 1 (bound by inclusion). *Let $P_{\text{vol}}^g \supseteq P^g$ be the set of loads that fit the trailer’s volume and the payload and clear the full-truckload floor on the binding resource, and let z_{vol} be the LP value of the master (44)–(47) over P_{vol}^g , certified as above. Then $z_{\text{vol}} \leq z_{3D\text{-SCPTOP}}^*$, and the same holds inside any region defined by bounds on the fleet vector.*

Proof. A valid placement satisfying (38)–(43) has total RTI volume at most Γ_K and total weight at most pK^w , and (42) is the same floor in both sets, so $P^g \subseteq P_{\text{vol}}^g$. The rows of the master — the linking, fleet, inventory and coverage rows, and any bound on n_{gt} or X — do not depend on the load model; only the column set does. The 3D master is therefore the volumetric master with columns removed, its LP value is at least z_{vol} , and its optimum is at least its LP value. $\square$

The practical consequence is the part that matters. A geometric pricing sweep costs $2|T|$ calls at the pricing limit and, on the benchmark instances, does not certify inside a root window at all; the volumetric relaxation — a two-constraint knapsack oracle over the same master — certifies in a few seconds. The method therefore takes its bound from the volumetric relaxation and asks geometric pricing only for columns. It can run with *no geometric pricing at the root at all*: the plan comes from the single-commodity pool, the construction and the tree, the bound comes from the relaxation, and the gap between them is a genuine optimality gap for the 3D problem. This is a principled division of labour rather than a shortcut, and Section 6 measures what root pricing adds when it is given time.

Rounding the duals. The prices π are irrational in general and must be rounded before entering an integer-coefficient subproblem. We round them *upwards*, so the rounded objective dominates the true one and the returned bound remains an over-estimate of ζ_{gt} , as the certificate requires. Rounding to nearest — the obvious choice — makes the bound a slight under-estimate and silently invalidates the certificate; with reduced costs at exactly zero it returns a value just above the threshold and convergence is never declared.

5.3 Pricing Subproblem

We solve the pricing problem with a constraint programming approach. The indices and parameters are the same as in the model proposed before, but the decision variables do not depend on the configuration index. Let c range over products when $g = p$ and over RTI types when $g = s$, let $\rho(c)$ be the RTI type carrying c , and let S be the slot bound (37). Table 4 describes the new decision variables.

Table 4: Nomenclature of the pricing subproblem

Decision variables	
u_i	Binary variable indicating whether slot i is used in the new load
χ_{ih}	Binary variable indicating whether slot i holds a stack of type h
ϱ_{ih}	Binary variable indicating whether that stack is rotated by 90°
δ_i^x, δ_i^y	Footprint of slot i along the x and y axes
x_i, y_i	Bottom-left coordinates of slot i on the truck floor
ℓ_{ir}	Number of layers of RTI type r in slot i
v_{ic}	Number of RTIs of commodity c loaded in slot i

a_c	Number of RTIs of commodity c in the new load
θ	Binary variable indicating whether the full-truckload requirement is met on volume rather than on weight

The objective is to find new truck loads with negative reduced cost. Dropping the constant $cK - \nu_{gt}$ of (49), that is

$$\max z = \sum_c \pi_{ct} a_c \quad (50)$$

subject to

$$\sum_{h \in H} \chi_{ih} = u_i, \quad \varrho_{ih} \leq \chi_{ih} \quad \forall i \leq S, h \in H \quad (51)$$

$$\delta_i^x = \sum_h (p_h \chi_{ih} + (q_h - p_h) \varrho_{ih}) \quad \forall i \leq S \quad (52)$$

$$\delta_i^y = \sum_h (q_h \chi_{ih} + (p_h - q_h) \varrho_{ih}) \quad \forall i \leq S \quad (53)$$

$$\text{NoOVERLAP2D} \left(\{[x_i, x_i + \delta_i^x] \times [y_i, y_i + \delta_i^y]\}_{i: u_i=1} \right) \quad (54)$$

$$x_i + \delta_i^x \leq pK, \quad y_i + \delta_i^y \leq qK \quad \forall i \leq S \quad (55)$$

$$\sum_{i \leq S} \sum_h p_h q_h \chi_{ih} \leq pK \cdot qK \quad (56)$$

$$\ell_{ir} \leq \lfloor rK/r_{gr} \rfloor \sum_{h: r \in R(h)} \chi_{ih} \quad \forall i \leq S, r \in R \quad (57)$$

$$\sum_{r \in R} r_{gr} \ell_{ir} \leq rK \quad \forall i \leq S \quad (58)$$

$$\sum_{c: \rho(c)=r} v_{ic} = n_{hr} \ell_{ir} \quad \forall i \leq S, r \in R \quad (59)$$

$$\sum_c w_c v_{ic} \leq W^{\max} \quad \forall i \leq S \quad (60)$$

$$a_c = \sum_{i \leq S} v_{ic} \quad \forall c \quad (61)$$

$$\sum_c w_c a_c \leq pK^w \quad (62)$$

$$\sum_c \gamma_{g\rho(c)} a_c \geq \alpha \Gamma_K \quad \text{if } \theta = 1 \quad (63)$$

$$\sum_c w_c a_c \geq \alpha pK^w \quad \text{if } \theta = 0 \quad (64)$$

$$\sum_{i \leq S} \sum_h p_h q_h \chi_{ih} \geq \alpha pK \cdot qK \quad \text{if } \theta = 1 \quad (65)$$

$$\sum_h h \chi_{ih} \geq \sum_h h \chi_{i+1,h} \quad \forall i < S. \quad (66)$$

Constraints (51)–(61) restrict the loads used in the pricing problem to feasible ones, reproducing (38)–(43). Equation (56) is a redundant area cut, and (65) its counterpart under the volume regime: a slot contributes at most (footprint area) $\times rK$ of RTI volume, so the utilisation floor implies a floor on the occupied floor area. The latter is what allows the solver to find a *first* feasible layout quickly, which for tight instances is harder than optimising once one is known. Constraint (66) breaks the symmetry among interchangeable slots by ordering them on the stack-type index.

The subproblem is stated as a constraint program because its hard core is the disjunctive non-overlap relation (54), which a CP solver propagates natively through a global constraint, whereas a MILP must encode it with the four big- M disjunctions (16)–(19) and accept the weak relaxation they entail. Whichever technology solves it, the oracle must solve *this* problem: an earlier MILP oracle imposed the full-truckload requirement as the sum of the two ratios in (42) rather than as their maximum, solved a strictly weaker problem, and returned a column that failed verification.

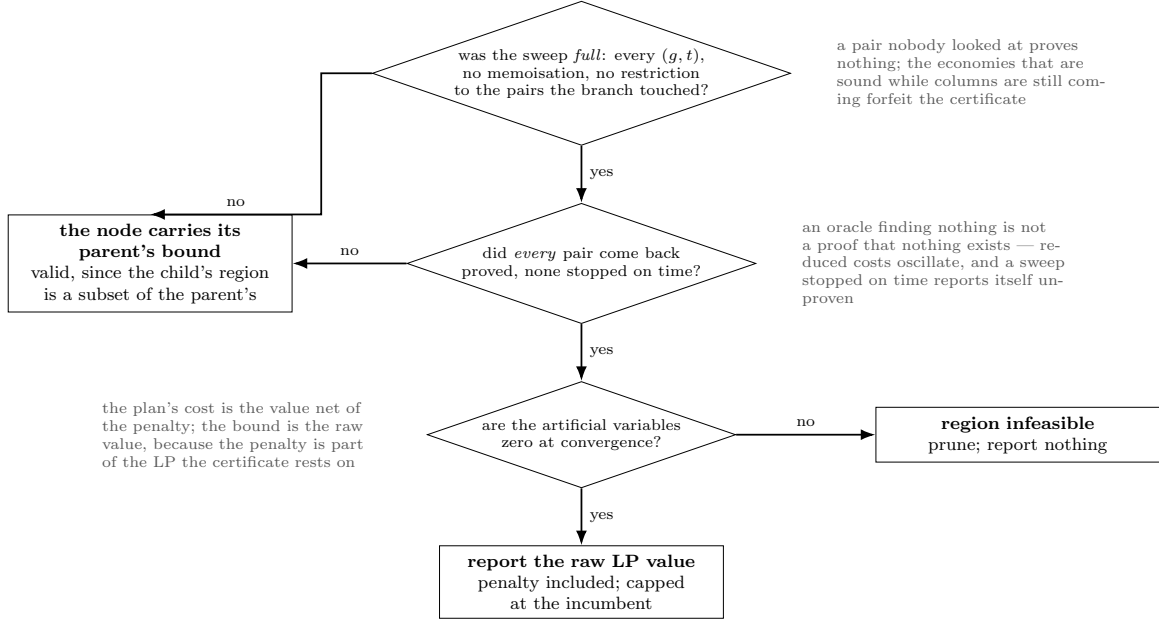


Figure 3: What may be reported as a lower bound. The restricted-master LP value is an *upper* bound on the master LP until pricing has converged, so it is used only as a pruning filter; a node obtains a bound of its own only from a full sweep in which every pair is proved, and otherwise inherits its parent's. A quantity above a solution one can exhibit is not a loose bound but a false statement, and it inflates every optimality gap computed from it, so the reported value is additionally capped at the incumbent.

The volumetric oracle. The same master, with the same inventory balances, coverage stock and fleet limit, is also solved over the loads of the volumetric benchmark of Appendix A: there a load is a column iff its volume fits the trailer's, its weight the payload, and it clears the full-truckload floor on the binding resource. Pricing is then a two-constraint integer knapsack that closes in milliseconds. Running both oracles through one master and one search is what makes the comparison of Section 6 an experiment about the loading representation rather than about two solution technologies, and it is the setting in which the search of Section 5.5 can afford to certify at every node.

What a certificate costs. Figure 4 shows the oracle and what it can and cannot return. A certified node is a full sweep, $2|T|$ calls each closed to proven optimality. On the volumetric oracle that is affordable at every node. On the constraint programme it is not — a single call on the benchmark instances takes tens of seconds to minutes, and a sweep is then the whole budget of a node — so with the CP oracle the certificate is taken from the volumetric relaxation (Proposition 1), at the root and in the one region that can refute the incumbent outright (Section 5.5), and geometric pricing is asked only for columns.

A pricing ladder. Any column of negative reduced cost improves the restricted master; the *most* negative one is needed only for the certificate. Column generation over the constraint programme therefore runs as a ladder. The first rung is a heuristic run *under the current duals* at every iteration — a randomised greedy construction over enumerated stack fills, placed by a shelf packer, together with a re-fill of the slot geometry of columns already in the pool — whose output is pricing output, never seeding. A heuristic load *qualifies* only if its reduced cost lies below a margin of half a percent of the pair's threshold, so that a marginal heuristic column cannot pre-empt a much better exact one; the best two qualifying loads per pair are kept, and a pair at which nothing qualifies goes to the exact oracle. The oracle itself is run in a stop-early mode that returns its first incumbent beating the threshold by the same margin. The certifying sweep never uses either rung: a certificate is one uncapped exact sweep, exactly as above, so the ladder changes which columns arrive and how fast, and nothing about what may be claimed. What each rung contributes is measured, not assumed, in Section 6.

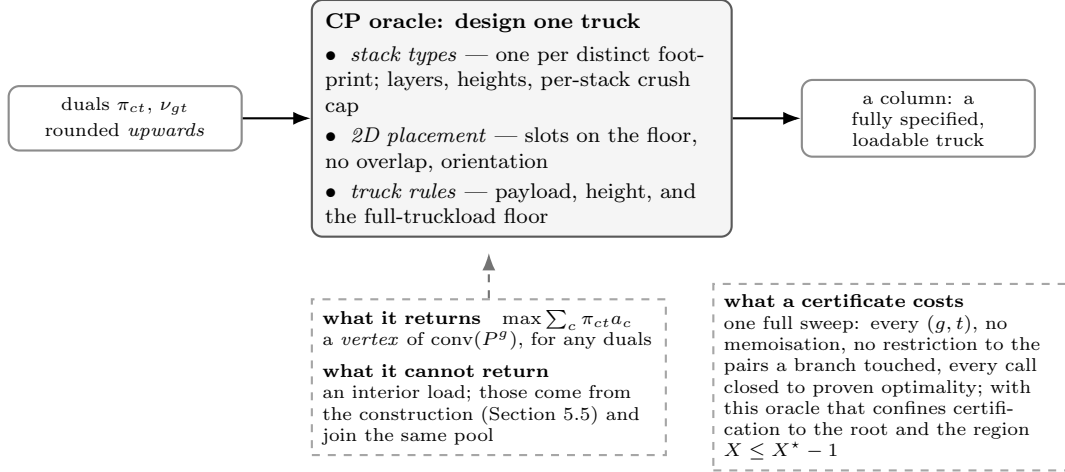


Figure 4: The pricing subproblem. Geometry, height, crush limits, payload and the full-truckload rule are enforced *inside* the oracle, so any column it returns is a physically loadable truck. It maximises the dual-weighted load and therefore returns a vertex of $\text{conv}(P^g)$ whatever the duals are; loads in the interior of that hull (Figure 5) are supplied by the construction of Section 5.5, not by pricing. A certificate needs every call of a sweep closed to proven optimality, which on this oracle is the whole budget of a node, and is therefore spent only where a bound changes what the tree can conclude.

5.4 Initial Columns

The search starts from the plainest possible pool: one maximally full truck per commodity and direction, obtained in the geometric case by running the pricing oracle for that commodity alone to proof — so the pool truck is the placeable maximum of the commodity, and the same number serves as its placeability cap below — and in the volumetric case by filling to capacity. No demand mixes, no diversification and no reduced variants are seeded, so whatever the search then reaches, it reached by pricing, branching and construction. This is a requirement of the method rather than a convenience: a pool assembled by hand contaminates every claim about what the search finds, and an earlier seeded pool was removed on that ground. The single-commodity pool guarantees that no direction is ever left without columns, and because these columns are shared they are offered to every period.

What such a pool, and any pool grown from it by pricing alone, *cannot* contain is shown in Figure 5 on a real pool. For any dual vector, (50) attains its optimum at an extreme point of $\text{conv}(P^g)$, so pricing returns extreme loads only. A balanced load lying inside the hull is dominated componentwise by a fractional blend of single-commodity trucks; the relaxation buys that blend for the price of one truck, while an integer plan must pay for each truck it mixes. No choice of duals returns the interior load, however the search moves them. Interior loads are precisely what an integer plan needs when a period’s demand does not decompose into full trucks, and the method therefore has a second generator for them, described next.

5.5 The fleet tree

Branching on the master’s own variables is the obvious rule and the wrong one here. Bounding one λ_{pt}^g removes one load from the relaxation, but a Dantzig–Wolfe master of this kind holds many columns that differ only in detail, so the relaxation answers by shifting the same fractional weight onto a near-identical column. Run to exhaustion on a benchmark instance with the pool frozen, that rule branched thousands of times without ever producing an integral relaxation. Branching on a per-period dispatch total has no symmetric twin, but it has an escape of its own: the relaxation answers a bound on one period by moving a truckload into a neighbouring period, at the price of one period’s holding cost, so the bound barely moves.

We therefore branch on the *fleet*. Let $n_{gt} := \sum_{p \in P^g} \lambda_{pt}^g$ be the dispatch total of direction g in period t , $\mathbf{n} = (n_{gt})_{g,t}$ the fleet vector, and $X := \sum_{g,t} n_{gt}$ the total fleet. Both are integer in every plan, and every bound placed on them is a row of the master that already carries a dual, so the pricing subproblem is untouched. Given an incumbent plan with fleet vector $\mathbf{n}^*$ and total X^* , the tree has two levels that ask different questions; Figure 6 draws them.

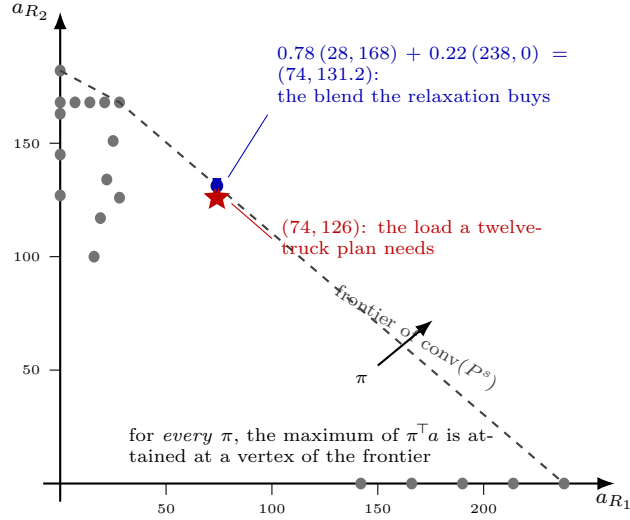


Figure 5: Why the load a cheaper plan needs is never priced. Dots are the supplier-bound loads obtained from a sweep of demand-derived price vectors; the dashed line is the upper-right frontier of their convex hull. The pricing objective is linear, so for *any* dual vector π its optimum is attained at a vertex of that frontier. The load $(74, 126)$ lies strictly below it — it is dominated componentwise by the blend $0.78(28, 168) + 0.22(238, 0) = (74, 131.2)$ — so no choice of duals returns it. The relaxation buys that blend for one truck, while an integer plan must pay for both of the trucks it mixes. The construction of Section 5.5 reaches the interior directly: it partitions a delivery vector into exact loads instead of asking a linear objective for them.

Level 1: the total fleet. Transport cost is exactly $cK \cdot X$, so a bound on the total has nothing to compensate it. Three regions partition the space:

- $X \geq X^* + 1$ costs a whole truck with no offset. A plan in it improves on the incumbent only if the extra truck saves more than cK minus the incumbent’s gap in inventory, and the region’s relaxation says at once whether that is possible; it is usually pruned by bound.
- $X = X^*$ fixes transport. Everything inside is inventory allocation, and this is where the second level goes.
- $X \leq X^* - 1$ is the only region that can beat the incumbent outright — “is there a plan with fewer trucks?” — and is therefore worth a certificate of its own. It is also the region the tree constructs in *first*: with the equality region on top of the stack its Murty subtree can take the whole budget while a smaller fleet is never asked for, which is what happened on a pure pool before the order was changed (Table 9).

A plan found *inside* an inequality region, with total X_2 , splits that region on its total exactly as the root was split — $[\cdot, X_2 - 1]$, $\{X_2\}$ and $[X_2 + 1, \cdot]$ cover it — and the equality piece starts a Murty chain of its own; level 1 is recursive. A construction that fails inside a region says only that the *pool* cannot support that fleet, since step 2 is exact over the pool; the region is therefore priced under its own duals with a small share of the budget and constructed once more before it is given up, and a bound proved by that pricing prunes it.

Level 2: the fleet vector, inside $X = X^*$. Fix an order on the pairs (g, t) — periods first, then direction. The fleet vector of any plan in the region differs from $\mathbf{n}^*$ at a first index j , or nowhere. Branching at the first pair where a solution may differ gives three children, $n_j \leq n_j^* - 1$, $n_j \geq n_j^* + 1$ and $n_j = n_j^*$, and the equality child is partitioned again at index $j + 1$ (Murty’s scheme). The regions are disjoint and exhaustive, and the equality child is expanded first so that the neighbourhood of the incumbent is searched before its complement. The partition is taken around the plan *constructed at that node*, not around the global incumbent, so a node that improves the incumbent is immediately re-partitioned around the better plan and nothing has to be restarted. The chain is cut at a fixed depth.

Processing a node. A node is a region imposed on the RMP as fleet rows. The LP over the current pool is solved first; being an over-estimate of the region’s LP it is a filter and never a bound (Section 5.2). If the filter allows pruning and the certification policy admits the node, column generation is run to a certificate; a certified value at or above the incumbent prunes the node, an infeasible region is discarded, and a certified value below the incumbent is recorded as that region’s bound. In every other case the node carries its parent’s bound. The construction below is then run *inside* the region — it must stay inside, or the children it induces partition around a point outside the node — and if it returns a plan cheaper than the incumbent the incumbent is replaced. Finally, if the region fixes X , the Murty children are generated around the constructed plan.

The construction. The primal heuristic is a flow-first construction in four steps. It decides first how much moves and how many trucks carry it, and builds the trucks afterwards.

1. *Restricted master.* The LP over the pool, inside the region, gives the duals and the fractional point the node stands on.
2. *Aggregate integer master.* On a fresh copy of the RMP the shipment quantities Q_{nt} , Q_{rt} and the dispatch totals n_{gt} are made integral, with the column variables left *continuous*, and the region’s bounds are imposed on the n_{gt} as well as on the columns. This is where the objective is actually decided: it chooses the flows and the fleet, and it does so without a single oracle call. A solution carrying artificial slack beyond a small tolerance means the region is infeasible over this pool and the node is abandoned; a small slack means the integer solve ran out of time short of a clean solution, and the partitions built from it are still valid loads that step 4 may combine with the pool. Under 3D loading the aggregate master also carries, per direction, period and footprint, an *integer* number of stacks with the crush limit and the height budget imposed per stack and the stacks’ floor areas summed against n_{gt} trailer floors (Section 5.7). The rows are necessary, since each truck’s own stack counts sum to a solution of them, and they let step 2 see the ceiling a linear stack-area row cannot: without them it asked the partition for a delivery that needed thirty stacks on a floor that holds twenty-nine and paid a truck for it.
3. *Partition.* For each direction and period, the delivery vector chosen in step 2 is *covered* by n_{gt} legal trucks found by a small MIP: integer parts summing to *at least* the vector in every commodity, each within the trailer’s volume and payload and clearing the full-truckload floor on the binding resource, ordered by weight to break the symmetry among parts, and minimising the excess measured in volume, the resource the floor and the trailer are measured in. A cover rather than an exact split, because step 2 chooses its flows over *continuous* column weights, and a fractional blend of legal loads need not be realisable as legal loads — a delivery vector can be one no number of full trucks carries exactly, while two full trucks carry it with a few units to spare. The cover is valid because every part is still a legal load and enters step 4 as a column, not as a fixed dispatch; a plan that over-delivers is feasible in the master, where the surplus is priced as holding. If n_{gt} parts cannot be formed, $n_{gt} + 1$ is tried, provided the region allows it. A timeout is not a proof: only a model that comes back infeasible says the cover is impossible.
4. *Integer master over pool plus partitions.* The parts are added to the pool as columns and the integer master is solved over the enlarged pool, warm-started with the partition itself. Its solution is the node’s plan, and its value the candidate incumbent.

Two generators, one pool. Pricing returns extreme points of $\text{conv}(P^g)$; the construction returns covers of a delivery vector, which are interior points that no dual vector reaches (Figure 5). A column is a feasible load, and a node’s constraints bound the multipliers λ , not the load, so a column found anywhere is valid everywhere: every column from either generator goes into the one shared pool, and a node that is pruned still leaves its columns behind. That is the mechanism by which the tree improves on the root: it is not that it reaches an integral relaxation — it essentially never does — but that each region it visits asks the construction for a plan under constraints the root never saw, and the loads that plan needs are added for every later node to use.

What the tree reports. The incumbent is the cheapest verified plan found by any construction. The root bound is the certificate obtained before any branching — under 3D loading, that of the volumetric relaxation (Proposition 1). The global bound of a branch and bound is the minimum over its *frontier*, not its root: a region nobody expanded contributes nothing, which is why a dive reports its root value however far it has gone. At the end of the budget the regions still open are therefore swept — filtered by their pool

LP and certified where the filter allows — and the global bound is the minimum over the certified bounds of the open regions, floored at the root bound and capped at the incumbent. Under 3D loading a region’s certificate is taken, like the root’s, from the volumetric relaxation with the region’s fleet rows applied, which Proposition 1 covers: the rows bound the fleet vector, not the load model, so the volumetric master over the region bounds the 3D master over it. A geometric sweep of $2|T|$ calls at the pricing limit cannot prove anything inside a region’s share of the budget and, on the runs of Section 6, bounded no region at all. Optimality is claimed *only* when the root is certified, no region remains open, no restart has broken the partition, and the incumbent meets the global bound. None of the runs reported in Section 6 meets these conditions, and the paper claims no proof of optimality anywhere.

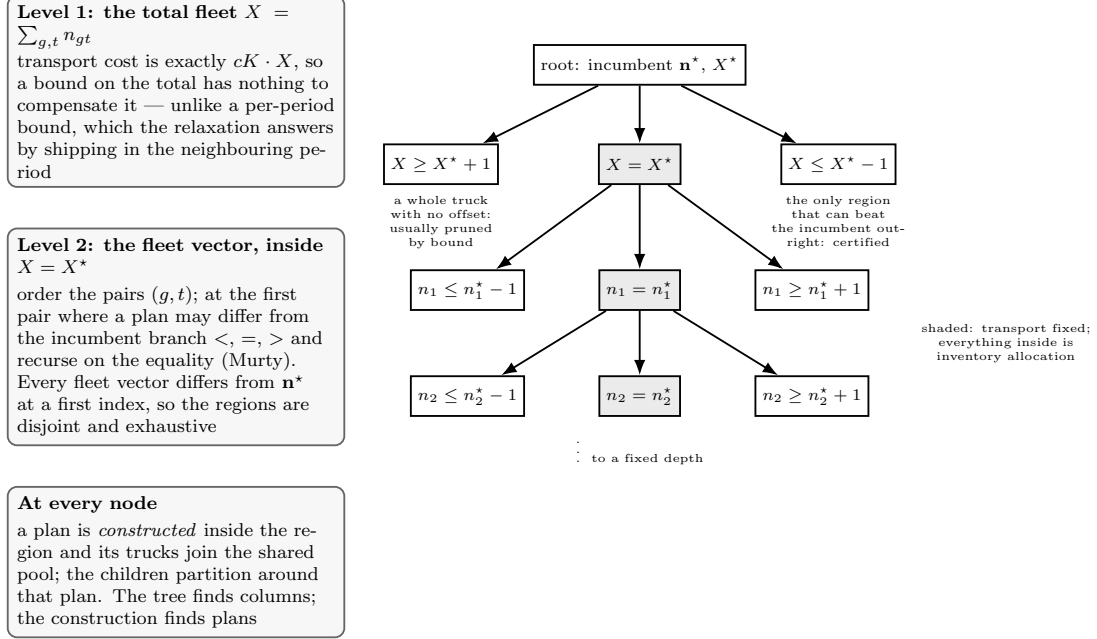


Figure 6: The two levels of the fleet tree. Level 1 partitions on the total fleet X , whose cost $cK \cdot X$ the relaxation cannot offset; level 2 partitions the region $X = X^*$ on the fleet vector itself, one pair at a time around the plan constructed at each node. Bounds on X and on n_{gt} are rows of the master that already carry duals, so the pricing subproblem is untouched, and a column found in any region is valid in every other.

5.6 Two corrections to the compact model

Implementing 3D-SCPTOP surfaced two points worth recording, both invisible at $lt = 1$.

Inventory recursion under a lead time. Constraints (2) and (9) carry inventory forward from period $t - lt$, whereas inventory accumulates one period at a time and only the *arrival* is delayed. They must read

$$I_{pnt} = I_{p,n,t-1} - d_{pnt} + Q_{n,t-lt} \quad \forall n \in N, t \geq lt + 1, \quad (67)$$

$$I_{estr} = I_{e,s,t-1,r} + Q_{r,t-lt} - \sum_{n \in M(r)} d_{snt} \quad \forall r \in R, t \geq lt + 1. \quad (68)$$

With $lt = 1$ the two versions coincide, which is why the defect survived; with $lt \geq 2$ the original silently discards $lt - 1$ periods of stock.

Coverage stock on folded RTIs. Constraint (12) requires $I_{etr} \geq ss_{etr}$ at *both* nodes, and every model in this paper enforces it that way: the master of Section 5.1, the volumetric benchmark of Appendix A, the company procedure PLC-SCPTOP and the compact model itself. An earlier implementation imposed the folded bound at the supplier only, on the argument that the plant has no operational use for a stock

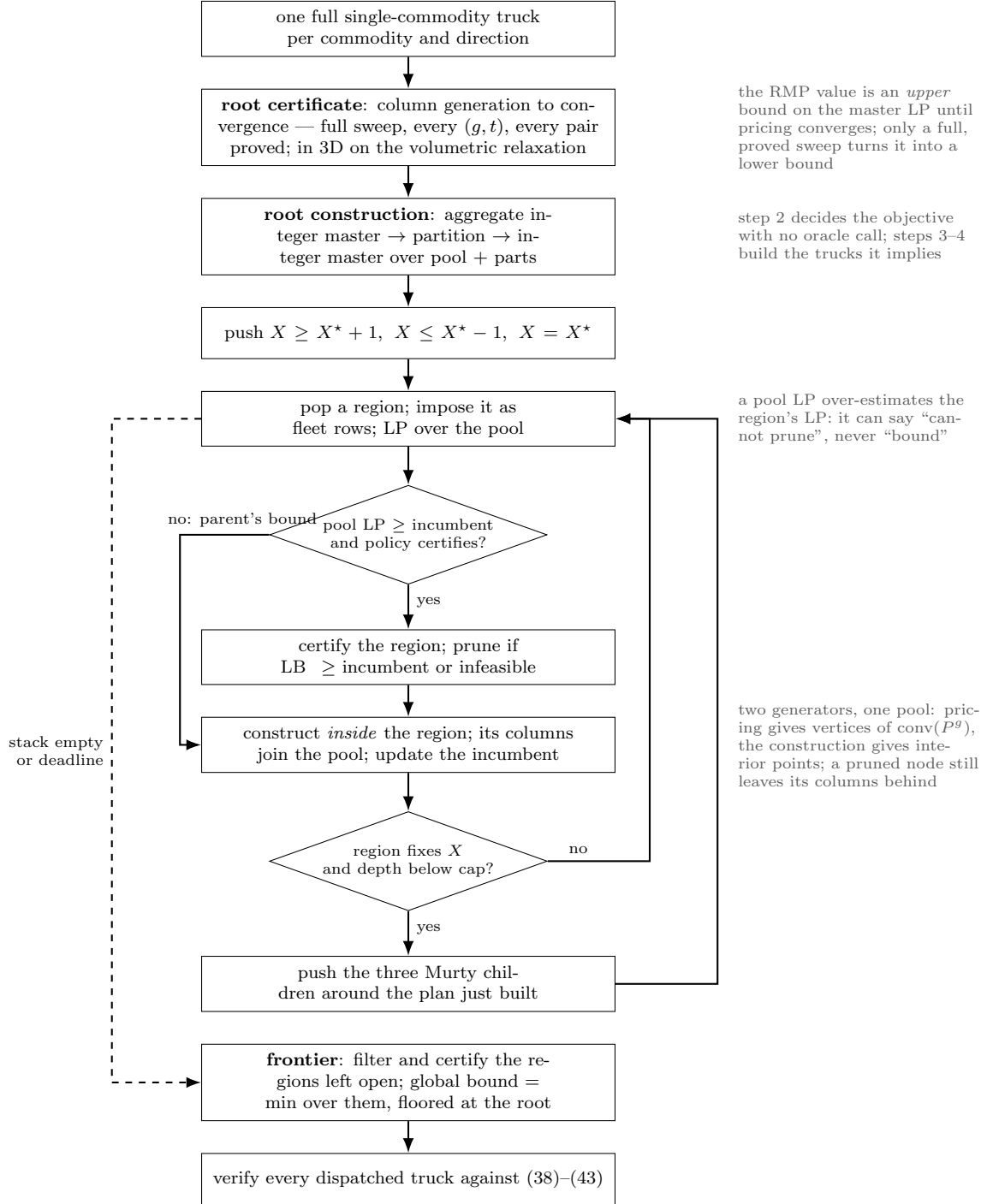


Figure 7: The fleet tree end to end. The search’s value is not that it reaches an integral relaxation — it essentially never does — but that each region asks the construction for a plan under constraints the root never saw, and the loads that plan needs are added to a pool every later node can use. The bound is decided by certification alone: a region that was not swept in full, with every pair proved, carries its parent’s bound, and the global bound is the minimum over the frontier rather than the root value.

of empties and that the extra row could render an instance with a tight RTI fleet infeasible. The row is

Algorithm 1 The fleet tree for 3D-SCPTOP: branch and bound over the fleet vector with the flow-first construction as primal

```

1:  $\bar{P} \leftarrow$  one full single-commodity truck per commodity and direction ▷ Section 5.4
2:  $LB_0 \leftarrow \text{CERTIFY}(\bar{P}, \text{root})$  ▷ full sweep, every  $(g, t)$ , every pair proved; else no bound. In 3D: on the volumetric relaxation, Prop. 1
3:  $(\bar{z}, \mathbf{n}^*, \bar{P}) \leftarrow \text{CONSTRUCT}(\bar{P}, \text{root}); \quad X^* \leftarrow \sum_{g,t} n_{gt}^*$  ▷ steps 1–4, Section 5.5
4:  $\mathcal{S} \leftarrow [X \geq X^* + 1, X = X^*, X \leq X^* - 1]$  ▷ a stack; the region that can beat the incumbent is expanded first
5: while  $\mathcal{S} \neq \emptyset$  and time remains do
6:    $\mathcal{R} \leftarrow \text{pop } \mathcal{S};$  impose  $\mathcal{R}$  on the RMP as fleet rows
7:    $\bar{z} \leftarrow \text{LP over } \bar{P} \text{ inside } \mathcal{R}$  ▷ over-estimates the region's LP: a filter, never a bound
8:   if LP infeasible then
9:     continue ▷ discarded
10:  end if
11:  if  $\bar{z} \geq \bar{z}$  and the policy certifies  $\mathcal{R}$  then
12:     $(LB_{\mathcal{R}}, \text{proved}) \leftarrow \text{CERTIFY}(\bar{P}, \mathcal{R})$ 
13:    if region infeasible or  $LB_{\mathcal{R}} \geq \bar{z}$  then
14:      continue ▷ pruned
15:    end if
16:    record  $LB_{\mathcal{R}}$  as an open bound
17:  else
18:     $\mathcal{R}$  carries its parent's bound
19:  end if
20:   $(z, \mathbf{n}, \bar{P}) \leftarrow \text{CONSTRUCT}(\bar{P}, \mathcal{R})$  ▷ inside the region; its columns join the shared pool
21:  if no plan then
22:    price under the region's duals; CONSTRUCT once more; continue if still none
23:  end if
24:  if  $z < \bar{z}$  then
25:     $(\bar{z}, \mathbf{n}^*) \leftarrow (z, \mathbf{n})$  ▷ new incumbent
26:  end if
27:  if  $\mathcal{R}$  does not fix  $X$  then
28:    push  $\mathcal{R} \cap \{X \geq X_2 + 1\}, \mathcal{R} \cap \{X = X_2\}, \mathcal{R} \cap \{X \leq X_2 - 1\}$  with  $X_2 = \sum_{g,t} n_{gt}$  ▷ level 1 recurses
29:  end if
30:  if  $\mathcal{R}$  fixes  $X$  and its depth  $j$  is below the cap then
31:     $(g, t) \leftarrow$  the  $j$ -th pair in the fixed order
32:    push  $\mathcal{R} \cap \{n_{gt} \geq n_{gt} + 1\}, \mathcal{R} \cap \{n_{gt} \leq n_{gt} - 1\}, \mathcal{R} \cap \{n_{gt} = n_{gt}\}$  onto  $\mathcal{S}$  ▷ Murty, around the plan just constructed
33:  end if
34:  release  $\mathcal{R}$ 
35: end while
36: for all  $\mathcal{R}$  still on  $\mathcal{S}$ , while a reserve of time remains do
37:   filter by the pool LP; certify the survivors; record their bounds ▷ the frontier
38: end for
39:  $LB \leftarrow \max\{LB_0, \text{min over open and frontier bounds}\},$  capped at  $\bar{z}$ 
40: verify every dispatched truck of the incumbent against (38)–(43)
41: return  $(\bar{z}, LB);$  optimality is claimed only if  $LB_0$  was proved,  $\mathcal{S} = \emptyset$ , no restart occurred and  $\bar{z} = LB$ 

```

what the model states, so it was reinstated, and the concern did not materialise: on the largest instances of Section 6 the root LP value is identical with and without it, no coverage row carries artificial slack, and the row is tight in a minority of periods without moving the optimum. Enforcing it everywhere also removes an asymmetry: had the master enforced the plant side and the comparison models not, the comparison would

have been unfair in the master’s favour.

5.7 The construction under 3D loading

Step 3 of the construction builds trucks from a delivery vector with a volumetric notion of a legal load. Over the volumetric oracle that notion *is* the definition of a column, and every part the partition proposes is a column by construction. Under 3D loading a load must also satisfy (38)–(43). The volumetric conditions are nevertheless *necessary* for geometric feasibility (Proposition 1), so the partition MIP of step 3 is kept as a *relaxation* of the geometric partition and each part it proposes is then tested for placeability. Step 2 makes no oracle calls and is unchanged, so the objective is still decided before any geometry is examined; what changes is only whether the trucks that objective implies can be built.

Deciding placeability. A part is submitted to a placeability oracle that returns one of three answers. A constructive *shelf design* — a small MIP that lays the part’s stacks in shelves along the trailer length, one integer count per shelf, footprint and orientation — supplies essentially every positive answer, and its layout is passed through the independent verifier of Section 6 before it counts; a witness the verifier rejects is not a witness. The constraint programme of Section 5.3, run in feasibility mode with the true slot bound (37), supplies the *infeasibility* proofs. A part left undecided within the limit is neither accepted nor refuted; retrying it longer does not help in practice, since such loads are hard rather than slow. A placeable part becomes a column with its layout as witness, exactly as a priced column does.

Two kinds of cut, and they are different claims. A part *proved* unplaceable yields a *dominance cut* forbidding every load that dominates it componentwise. The cut is valid because placeability is downward closed: removing an RTI from a placeable layout leaves every position unchanged and every stack no taller and no heavier, so a load dominating an unplaceable one is itself unplaceable, and the cut removes the whole up-set of the refuted load rather than a single point. The full-truckload floor is not part of the placeability test, so it does not break the closure. A part left *undecided* is instead excluded as a single point, so that the partition MIP proposes something else. That is a restriction of a primal heuristic and never a validity claim: it cannot make the method report an invalid plan, only fail to find one. The two constraints are kept separate because they rest on different grounds, and only the first may ever be read as knowledge about the instance.

Placeability caps. The maximum count of a single commodity in one truck is a scalar, and by the same closure it is monotone, so it is bisected once per direction and commodity at the start of a run — a proved pricing optimum tightens it, an undecided probe leaves the analytic cap in force — and handed to the partition MIP as an upper bound before it solves. Without it a single-commodity dominance cut advances one unit per round, from the volumetric fill down towards the placeable count, at a placeability call and a partition re-solve each. The caps also measure the volumetric relaxation’s optimism directly: on *I_16.15.2.90_k25* the payload admits 57 full RTIs of product P_1 in one truck and 32 can be placed, because its 650 mm container stacks four high under the roof and two high under the crush limit.

A stack relaxation inside the partition. The partition MIP additionally carries, per truck and footprint, an integer number of stacks, with the crush limit and the height budget imposed per stack and the stacks’ floor areas summed against the trailer floor; summed over the n_{gt} trucks of a period the same rows sit inside the aggregate master of step 2, with one integer count per footprint and period. It is valid — every placeable load satisfies it, under the crush rule (41) that bounds the total weight of a stack — and it is exactly the constraint the volumetric relaxation omits: a family whose RTIs are short enough for four layers by height but heavy enough for only two by the crush limit is credited by volume with twice its placeable count. The relaxation says nothing about the two-dimensional arrangement of the stacks, which the placeability test and the dominance cuts cover.

The pool and the bound. The starting pool is the placeable single-commodity trucks of Section 5.4 and nothing else; every other column comes from the pricing ladder or from the construction. On a catalogue where the crush limit binds, the restricted master over that pool is not even feasible — no combination of pure trucks covers demand within the fleet — and reaching feasibility is the heuristic rung’s job (Table 10). The root bound is the certified value of the volumetric relaxation (Proposition 1); geometric pricing, given a share of the root budget, supplies columns and nothing else. Section 6 reports what it supplies.

6 Computational Experiments

The purpose of the experiments is not to establish optimality. For procurement transport planning at an operational level, what matters is whether a procedure returns, in a usable amount of time, a plan that a loading crew can actually execute and that improves on what the company does today. The experiments are organised around those two questions, and a third that turns out to dominate both: whether a model that abstracts the loading geometry away produces plans that can be executed at all.

6.1 Protocol and declared budgets

All calculations use Python. The restricted master, the aggregate integer master and the partition MIPs of the construction, the volumetric benchmark and the compact models are solved with Gurobi 12; the pricing subproblem (50)–(66) and the placeability tests with CP-SAT from OR-Tools 9.12. The 3D runs of Tables 7–10 and the compact references they are set against were made on one machine (Gurobi 12.0.1, Python 3.12.3, ten solver threads on a six-core, twelve-thread desktop, so the thread count over-subscribes the physical cores and is reported as the value actually used), each from a recorded run that names its commit. No other solver job shared the cores while any reported row ran, with one exception recorded in Table 9’s notes; the machine of the regenerated campaign is recorded with each run and reported with its tables ([**pending: host and core count of the regenerated campaign**]).

Every row carries its configuration. Five parameters can change the answer a solver returns, and every run records all five with its result: the wall-clock *time limit*; the relative *MIP gap* at which an integer solve is allowed to stop; the *thread count*; the *memory cap* per model; and the random *seed*. Table 5 states the values under which each stage of the study is run. A row whose configuration is not recorded is not reported: an earlier set of fleet-tree runs was discarded on exactly that ground, since nothing tied them to a budget. Two families of auxiliary files are exceptions and are labelled as such where they are used: the offline fleet-floor certificates (Table 11) and the stand-alone ladder measurement (Table 10) record their own limits, threads and commit but not the memory cap or the seed, neither of which binds a single-threaded certification or a fixed-iteration measurement.

Table 5: Declared budgets, per stage. Every value is read from the run’s own record, not from the script that launched it; the memory cap is Gurobi’s per-model soft limit, at which a solve stops and keeps its incumbent and dual bound. The fleet tree divides its time limit between phases as described in Section 5.5: in 3D, 15% of the budget to root column generation, 3% reserved for the frontier sweep, placeability calls limited to 5 s and pricing calls to 20 s (eight CP-SAT workers each), the heuristic rung on with a 0.5% qualifying margin and the stop-early oracle, an initial Murty depth of 6, at most two cut-and-repack rounds per partition attempt, at most 15% of the budget on the cheap volumetric region certificates, and the expensive geometric sweep only at the region $X \leq X^* - 1$. The depth is initial rather than fixed: when the stack empties with budget left the cap rises by two and the leaves it had stopped at are re-opened, and when the search then exhausts, what remains of the budget goes to one exact geometric certification over the final pool, whose time is reported separately. Rows at a 15% frontier reserve and a fixed depth are kept in Table 9 as the before-picture. The 900 s budget applies to *I_8.5.2.90* and *I_8.10.2.90_k25*, 1 800 s to *I_12.12.2.90_k25* and *I_16.15.2.90_k25*, and 3 600 s to the second *I_16.15.2.90_k25* row and to *I_24.20.2.90_k50*.

stage	time limit (s)	MIP gap	threads	memory cap (GB)	seed
fleet tree, volumetric master (Alg. 1)	[pending]	[pending]	[pending]	[pending]	[pending]
compact SCPTOP reference	[pending]	[pending]	[pending]	[pending]	[pending]
MATH-3D-SCPTOP, 3D (Tables 7–10)	900 / 1 800 / 3 600	$5 \cdot 10^{-4}$ (step 2), 0 (step 4)	10	20	0
compact 3D-SCPTOP, $ L = 3$ (Table 7)	1 800	0	10	20	0
compact SCPTOP, volumetric reference of the 3D rows (Table 8)	3 600	0	10	20	0
MATH-3D-SCPTOP, CP oracle (benchmark)	[pending]	[pending]	[pending]	[pending]	[pending]
SCPTOP (volumetric benchmark)	[pending]	[pending]	[pending]	[pending]	[pending]
PLC-SCPTOP (company procedure)	[pending]	[pending]	[pending]	[pending]	[pending]
placeability recheck of SCPTOP loads	[pending]	—	[pending]	[pending]	[pending]

Memory is a budget, not an accident. Gurobi holds its branch-and-bound tree in memory and imposes no limit of its own, so a compact model whose dual bound has stalled simply grows its list of

unexplored nodes until the machine intervenes. We therefore give every model an explicit per-model soft memory limit, at which the solver stops and returns its incumbent and its dual bound with the status `MEM_LIMIT`, exactly as a time limit would. Two consequences follow for how the results may be read. A solve that stopped on memory is a solve that stopped, and its dual bound is a valid lower bound on the model’s optimum regardless of why it stopped. And reaching a fixed per-model cap sooner on a larger model is a statement about the cap, not about tractability; we draw no inference from *when* the cap was reached, and the cap is reported as a parameter of the experiment like the time limit.

What each reported quantity means.

- z is the cost (44) of the plan returned, net of any artificial penalty; a plan is reported only if it carries no artificial slack. For the fleet tree and MATH-3D-SCPTOP it is the incumbent of Algorithm 1; for the compact models it is the solver’s incumbent at its stop. It is an upper bound on the optimum in every case.
- $trucks$ is $\sum_{g,t} n_{gt}$ for that plan.
- LB is a lower bound on the optimum of the model the method solves. For the tree it is the *certified* bound of Section 5.2: the root certificate, or the minimum over the certified bounds of the open frontier when that is higher. A *root proved* flag records whether the root sweep returned every pair proved; without it no bound is printed. For a compact model it is the solver’s dual bound at its stop.
- gap is $(z - LB)/z$ against the method’s own bound. It is an optimality gap only where the bound is certified; it is never a gap to a proven optimum unless the row says *proved*.
- $stop$ names the condition under which a solve ended: `OPTIMAL`, `TIME_LIMIT` or `MEM_LIMIT`. A time or memory stop makes the row’s time the budget rather than a solve time.
- $nodes$, $open$, $columns$ (priced / built) and the time split between column generation, the aggregate master, the partitions and the integer master describe the tree. *open* is the number of regions left unexpanded when time ran out; a run with open regions has not searched the space, whatever its gap.
- $loadable$ is the outcome of re-verifying every dispatched truck of a plan against (38)–(43) by a routine that shares no code with the solver that produced it. *unloadable* counts, among the loads a volumetric plan dispatches, those a constraint programme has *proved* cannot be stacked and placed in a trailer; loads the programme could not decide within its limit are reported separately as *undecided* and are counted neither way.
- Δz is the cost of one method relative to another on the same instance, and is meaningful only between plans both of which are loadable.

6.2 Instance generation

Thirty-six benchmark instances are generated as a full factorial over the four explanatory variables of the study: the number of products $|N| \in \{4, 8\}$, the horizon $|T| \in \{5, 10\}$, the number of RTI types $|R| \in \{2, 3, 4\}$ and the volumetric granularity $v \in \{60, 90, 120\}$. Instances are named $I_{-|N|_{-}|T|_{-}|R|_{-}v}$.

- **Container dimensions:** footprints are drawn from the industrial catalogue $\{1200 \times 800, 1200 \times 1000, 800 \times 600, 1200 \times 1600, 2400 \times 1200\}$ mm. Heights are quantised to 50 mm and a footprint is admissible for a given v only if the implied erected height lies in $[400, 1500]$ mm, so that an RTI is worth stacking and at least two layers fit under the trailer roof. Whenever $|R| \geq 3$ several RTI types share a footprint, reproducing a real catalogue in which container families sit on a common pallet base; this is what allows a stack to mix types across layers.
- **Container volume:** v is the number of erected RTIs that fill one truck by volume, and every RTI type is generated with volume at least Γ_K/v . Larger v means smaller RTIs and a harder placement subproblem.
- **Demand:** generated per product and period by deviating $\pm 10\%$ from a mean calibrated so that roughly 2.5 truckloads are consumed per period. Calibrating the mean keeps difficulty comparable across the factorial instead of letting it drift with the RTI size.
- **Weights:** the content weight of a full RTI scales with its volume and varies by $\pm 25\%$ between products sharing an RTI type, which is what makes the per-stack crush limit depend on the period’s product mix rather than on the RTI type alone.

The truck is a standard semi-trailer, $pK \times qK \times rK = 13600 \times 2400 \times 3000$ mm, payload $pK^w = 24$ t, per-stack crush limit $W^{\max} = 1200$ kg, minimum utilisation $\alpha = 0.55$ and cost $cK = 1000$ per dispatch. Lead time is $lt = 1$ and coverage stock one period of demand.

A size ladder. The benchmark tops out at $|N| = 8$ and $|T| = 10$. To find where, if anywhere, the decomposition pays against the compact model, a second set of instances is generated with the same rules along two axes: $(|N|, |T|) \in \{(8, 10), (12, 12), (16, 15), (24, 20), (32, 20)\}$, and the number of truckloads consumed per period, 2.5, 5 or 8, which scales the fleet K and with it the number of interchangeable trucks the compact model indexes. The suffix is that figure times ten and is *not* K : $I_{8.10.2.90.k25}$, $I_{12.12.2.90.k25}$ and $I_{16.15.2.90.k25}$ have $K = 12$, and $I_{24.20.2.90.k50}$ has $K = 21$. These instances have $|R| = 2$ and $v = 90$ and are named $I_{|N|.|T|.2.90.k}$ followed by ten times the truckloads per period. They stress different things: $|N|$ and $|T|$ grow the master and the number of pricing calls, while the fleet grows the compact model’s per-truck variables and its symmetry.

Sizing the RTI fleet. One generation parameter deserves comment because it is easy to get wrong and the symptom is misleading. Empties return by the truckload, and a return truck leaves only once the plant has accumulated enough folded RTIs to clear the full-truckload floor. Since folded RTIs pack very densely, that takes several periods, during which the supplier must live off its own folded stock. If the fleet is sized only for the lead time, the linear relaxation remains feasible by dispatching fractional trucks while the integer problem has no solution at all. We therefore size the supplier’s initial folded stock of each type to cover $lt + SC$ periods plus the number of periods, capped at six, needed to accumulate $\lceil \alpha \Gamma_K / \gamma_{sr} \rceil$ folded RTIs at the plant’s rate of emptying them, which is the standard closed-loop argument that an RTI fleet must cover the cycle time.

6.3 Methods compared

- **Fleet tree** — Algorithm 1 on the Dantzig–Wolfe master over the volumetric oracle of Section 5.3, certified at the root and at the region $X \leq X^* - 1$. This is the configuration under which the size ladder is run, since it is where per-node certification is affordable.
- **Compact SCPTOP reference** — the volumetric model of Appendix A solved directly by Gurobi, under the declared budgets of Table 5. It solves the same problem as the fleet tree row above, so the two bracket a common optimum.
- **MATH-3D-SCPTOP** — Algorithm 1 on the 3D problem, Section 5.7: a starting pool of the placeable single-commodity trucks and nothing else, columns from the pricing ladder of Section 5.3, the volumetric partition kept as a relaxation and run as a cover, every part proved placeable before it becomes a column, dominance cuts on the parts proved unplaceable, integer stack counts in the aggregate master, the region $X \leq X^* - 1$ explored first, and the bound from the volumetric relaxation (Proposition 1). Every dispatched truck is verified.
- **Company procedure (PLC-SCPTOP)** — the master problem over a fixed pool of manually derived loading configurations, one regular grid per container type, as produced today by a spreadsheet.
- **SCPTOP** — the volumetric benchmark of Appendix A, identical to the master in every respect *except* that truck capacity is volumetric. The comparison therefore isolates the effect of the loading representation.
- **Compact 3D-SCPTOP** — solved directly, on the small instances where that is still possible, warm-started with the matheuristic’s plan.

Every truck that a geometric method dispatches is re-verified from scratch against (38)–(43) by a routine that shares no code with the solver that produced it. Every truck that SCPTOP dispatches is submitted to an independent constraint programme asking whether that exact set of RTIs can be stacked and placed in a trailer at all, with the slot bound (37) in force so that an infeasibility verdict is a proof.

6.4 Results

Tables 7–10 hold the 3D results, each read from a recorded run whose file is named beside the row in the source; cells marked pending name the run that will fill them. **[pending: The benchmark, case-study, oracle and exact-comparison tables of Tables 12–18 and the volumetric ladder of Table 19 are being regenerated under the declared budgets of Table 5. No number from the previous revision is carried forward.]**

Table 6: MATH-3D-SCPTOP on the size ladder, **every row on one commit** (73411b9; the optimality proof of footnote ^b is a separate run on 11db34b), ten threads, a 20 GB per-model cap and seed 0, run sequentially. *nodes* is the tree nodes opened, *dropped* the regions left with no bound covering them, *open* those still on the stack at the deadline; a region counted *closed* carries a certified bound, which is not the same as being refuted (see Section 6). *root* is the plan the construction returns before any node is popped and *tree* the improvement the search then makes on it, both read from the same run’s trace: this is the controlled measurement of what branching contributes, and it replaces a cross-commit comparison of two verified upper bounds, which is not one. All plans are verified truck by truck by code sharing nothing with the solver. The bound is the certified volumetric relaxation, valid for 3D by Proposition 1; it is loose wherever the crush limit binds, so the gap on the last two rows is not an optimality gap. *I_8_5_2_90* is the exception and is the paper’s only proved optimum: there the volumetric relaxation itself solves to optimality at exactly the 3D plan’s cost (footnote ^b).

Instance	z	trucks	verified	LB (cert.)	gap	root (t)	tree	nodes	dropped	open	wall / budget (s)
<i>I_8_5_2_90</i>	18 796.96 ^b	12	12 / 12	18 796.96 ^b	0 (optimal)	18 796.96 (140 s)	0.00%	67	0	0	873 / 900
<i>I_8_10_2_90_k25</i>	38 855.34	26	26 / 26	38 851.9 ^c	0.009%	38 865.30 (136 s)	0.03%	68	0	35	883 / 900
<i>I_12_12_2_90_k25</i>	52 129.45	34	34 / 34	52 105.2	0.046%	53 195.84 (459 s)	2.00% ^d	11	0	10	1 761 / 1 800
<i>I_16_15_2_90_k25</i>	78 549.14	54	54 / 54	68 139.2 ^c	13.3% ^e	78 549.14 (1 181 s)	0.00%	1	0	2	1 746 / 1 800
<i>I_24_20_2_90_k50</i>	230 594.36	160	160 / 160	200 013.4	13.3% ^e	230 594.36 (1 418 s)	0.00%	2	0	4	3 492 / 3 600

^b The volumetric relaxation of this instance solves to proved optimality at 18 796.96 in 0.18 s, and the verified 3D plan costs the same to machine precision. Every placeable load is volumetrically feasible, so $z_{3D}^* \geq z_{vol}^*$ and a feasible 3D plan attaining it is optimal. *This proof does not come from the method*: no tree, no pricing, no geometric certification. It is available because on this catalogue the height-limited and crush-limited layer counts coincide, which is precisely the condition under which Section 6 argues a volumetric capacity is adequate. The method’s own attempt at the same proof — certifying, under their own duals, the three regions its search could not refute — failed, at 237 s per region. ^c From the compact volumetric model’s dual bound, tighter than the method’s own certificate and valid for 3D by inclusion. ^d This is where branching does the most work in the paper: the construction returns 53 195.84 at 459 s and five node improvements take it to 52 129.45, a 2.00% gain that reproduces under a second certification policy (53 248.76 $\rightarrow$ 52 127.89). A run on an earlier revision reached 52 123.03, 0.012% below this row; that is a comparison of two verified upper bounds across commits and we report it here rather than in the table, because it is not a controlled experiment. ^e Not an optimality gap. The volumetric bound is loose exactly where the crush limit halves the layer count, which is the regime these two instances were generated to expose.

Table 7: **Earlier code revisions** for the MATH-3D-SCPTOP columns, retained for the compact-model comparison, which has not been re-run; the current method is Table 6 and its numbers differ. The compact 3D-SCPTOP model ($|L| = 3$, Gurobi) against MATH-3D-SCPTOP on the size ladder in 3D. Both ran on the same machine with ten threads, a 20 GB per-model memory cap and seed 0. **The budgets differ on some rows:** the compact model had 1 800 s throughout, the tree 900 s on *I_8.5.2.90* and *I_8.10.2.90_k25*, 1 800 s on *I_12.12.2.90_k25* and *I_16.15.2.90_k25*, which are the equal-budget pairs, and 3 600 s on the last two rows. The *I_8.5*, *I_8.10* and *I_12.12* tree rows are on the final code, in which a region’s certificate comes from the volumetric relaxation, with the frontier reserve at 3% of the budget, so that the three share one commit and one knob set; the *I_16.15* rows predate it. On *I_8.10* the run on the preceding commit, same cap, returned a plan 0.002% better, 38 855.06, and the text says so (Section 6). *stop* is the condition under which the compact solve ended and its LB its dual bound at that stop; *none* means no feasible solution was found within the budget. The tree’s LB is the certified bound of the volumetric relaxation (Proposition 1), its *gap* is to that bound, and *first hit* is the time at which the reported plan was found. *verified* is the independent truck-by-truck check, and *commit* the code revisions (compact / tree) each row ran on. No row is an optimum; the last column is the interval in which the optimum lies, from the cheaper of the two plans and from the tighter bound *that is a bound on the problem*: the compact model restricts each direction to $|L| = 3$ configurations a priori, so its dual bound bounds that restriction and not z^* , and it is reported in its own column but never as an end of the interval. Where a compact *volumetric* solve exists its dual bound is used instead, being valid for 3D by Proposition 1.

Instance	compact 3D-SCPTOP, 1 800 s						MATH-3D-SCPTOP									
	z	trucks	LB	stop	RSS (GB)	verified	z	trucks	LB (cert.)	gap	budget (s)	first hit (s)	nodes / open	verified	optimum in	commit
<i>I_8.5.2.90</i>	18 816.13	12	18 795.76	TIME_LIMIT	3.8	12 / 12 ^a	18 796.96	12	18 795.54	0.0076%	900	185.4	147 / 7	12 / 12	[18 795.54, 18 796.96]	4f66dc6 / 4f66dc6
<i>I_8.10.2.90_k25</i>	<i>none</i>	—	36 458.75	TIME_LIMIT	6.1	—	38 855.70	26	38 791.14	0.17%	900	481.9	30 / 5	26 / 26	[38 851.93, 38 855.06] ^c	647d684 / 4f66dc6
<i>I_12.12.2.90_k25</i>	<i>none</i>	—	48 516.86	TIME_LIMIT	2.7	—	52 123.03	34	52 105.25	0.034%	1 800	1 691.9	17 / 11	34 / 34	[52 105.25, 52 123.03]	bdd39ca / 4f66dc6
<i>I_16.15.2.90_k25</i>	<i>none</i>	—	64 053.23	TIME_LIMIT	4.5	—	78 808.39	54	68 131.43 ^b	13.5% ^a	1 800	1 530.0	0 / 3	54 / 54	[68 139.17, 78 808.39]	bdd39ca / 9c33c68
<i>I_16.15.2.90_k25</i> , 3 600 s			as above (1 800 s)				78 730.33	53	68 131.43 ^b	13.5% ^a	3 600	1 815.4	1 / 2	53 / 53	[68 139.17, 78 730.33]	— / bdd39ca
<i>I_24.20.2.90_k50</i>			not run				<i>no plan</i>	—	200 013.41	—	3 600	—	0 / 0	—	[200 013.41, —]	— / bdd39ca

^a From the re-run of the compact model after a defect in the layout extraction was fixed; the first run returned the same objective, bound and stop condition, with no phantom freight (*leaked* = 0), but its extracted layouts stated under-filled slots against the designed layer count and failed the verifier. The re-run’s twelve loads pass it. ^b The volumetric bound is loose on this catalogue, where the crush limit halves the layer count of the R_1 family; see the text. The LP-certified value is 68 131.43; the compact volumetric model’s proven optimum on the same instance, 68 139.17, is also a valid 3D bound by inclusion (Table 8) and is the lower end of the interval. ^c Upper end from the best verified plan on this instance, 38 855.06 (Table 8); lower end from the compact volumetric model’s dual bound, 38 851.93, which is valid for the 3D problem by Proposition 1 and is tighter than the method’s own certified 38 791.14.

Table 8: MATH-3D-SCPTOP against the compact volumetric model (SCPTOP, Appendix A) on the same instances. The volumetric model is the relaxation: every placeable load is volumetrically feasible, so a verified 3D plan is a feasible SCPTOP solution and can never be cheaper than the volumetric optimum, and the volumetric dual bound is a valid bound for the 3D problem (Proposition 1). The compact volumetric solves had 3 600 s, ten threads and a 20 GB cap; *stop* names how each ended and its LB is its dual bound at that stop. The 3D rows are those of Table 7; *used* is the wall time of the run, *parts* the share of distinct compositions the partition proposed that were placed, proved unplaceable, or left undecided. *Gap* is to the 3D method’s own certified bound, never to an optimum. The *I_8.10.2.90_k25* row is the best verified plan on that instance, from the run on the commit preceding the final code with the same knobs and cap. On *I_24.20.2.90_k50* the method certified its bound and returned no plan within the budget; the row records that.

Instance	compact volumetric (SCPTOP), 3 600 s				MATH-3D-SCPTOP						
	z	LB	stop	trucks	z	trucks	LB (cert.)	gap	budget / used (s)	nodes / open	parts placed / unplaceable / undecided
<i>I_8.10.2.90_k25</i>	38 855.65	38 851.93	MEM_LIMIT	26	38 855.06	26	38 791.14	0.16%	900 / 900	39 / 12	96.3% / 0.0% / 3.7%
<i>I_12.12.2.90_k25</i>	[pending]	[pending]	[pending]	[pending]	52 123.03	34	52 105.25	0.034%	1 800 / 1 770	17 / 11	80.1% / 4.5% / 15.4%
<i>I_16.15.2.90_k25</i>	68 139.17	68 139.17	OPTIMAL	45	78 808.39	54	68 131.43	13.5%	1 800 / 1 530	0 / 3	66.9% / 31.2% / 1.9%
<i>I_16.15.2.90_k25</i> , 3 600 s	68 139.17	68 139.17	OPTIMAL	45	78 730.33	53	68 131.43	13.5%	3 600 / 3 060	1 / 2	57.6% / 40.5% / 1.9%
<i>I_24.20.2.90_k50</i>	200 266.21	199 518.04	MEM_LIMIT	137	<i>no plan</i>	—	200 013.41	—	3 600 / 3 065	0 / 0	60.4% / 36.8% / 2.8%

Table 9: What the starting pool, the search order and the frontier reserve cost (ten threads, seed 0). The first row seeded the pool by hand with demand-mix and pair fills and is *not* the method; it is the reference the pure-pool rows are measured against, and it ran at a 4 GB cap on an earlier commit, so it is not a controlled comparison with them. The pure rows start from the single-commodity trucks and nothing else. *integer stacks* is whether step 2 counts stacks per footprint in integers; *order* is which level-1 region the tree constructs in first; *reserve* is the share of the budget held back for the frontier sweep. Rows marked *final* are the final code, in which a region’s certificate comes from the volumetric relaxation; on it the sweep takes seconds, so a 15% reserve is unused time, and the 3% rows are the ones Table 7 reports. Every plan is verified truck by truck; the bound is the certified volumetric relaxation in every row.

Instance	pool	rung	integer stacks	order	reserve	cap (GB)	z	trucks	first hit (s)	budget (s)	nodes / open	LB (cert.)
<i>I.8.5.2.90</i>	seeded by hand (reference)	no	no	$X = X^*$ first	15%	4	18 796.96	12	48.5	900	219 / 0	18 795.54
<i>I.8.5.2.90</i>	pure	no	no	$X = X^*$ first	15%	4	19 682.36	13	172.2	900	170 / 10	18 795.54
<i>I.8.5.2.90</i>	pure	yes	no	$X = X^*$ first	15%	20	19 682.36	13	140.3	900	180 / 9	18 795.54
<i>I.8.5.2.90</i>	pure	yes	yes	$X \leq X^* - 1$ first	15%	20	18 796.96	12	139.9	900	170 / 5	18 795.54
<i>I.8.5.2.90</i>	pure, final	yes	yes	$X \leq X^* - 1$ first	15%	20	18 796.96	12	140.2	900	135 / 4	18 795.54
<i>I.8.5.2.90</i>	pure, final	yes	yes	$X \leq X^* - 1$ first	3%	20	18 796.96	12	185.4	900	147 / 7	18 795.54
<i>I.8.10.2.90.k25</i>	pure, final	yes	yes	$X \leq X^* - 1$ first	15%	20	38 859.44	26	420.8	900	18 / 7	38 791.14
<i>I.8.10.2.90.k25</i>	pure, final	yes	yes	$X \leq X^* - 1$ first	3%	20	38 855.70	26	481.9	900	30 / 5	38 791.14
<i>I.12.12.2.90.k25</i>	pure, final	yes	yes	$X \leq X^* - 1$ first	15%	20	52 129.91	34	1 515.3	1 800	12 / 9	52 105.25
<i>I.12.12.2.90.k25</i>	pure, final	yes	yes	$X \leq X^* - 1$ first	3%	20	52 123.03	34	1 691.9	1 800	17 / 11	52 105.25

Table 10: The pricing ladder. Top: the heuristic rung measured against the exact oracle under the *same duals* over the pure pool, in a stand-alone column-generation loop with two threads, two CP-SAT workers and a 5 s exact oracle — a measurement of which columns each finds, not of production speed. *LP* is the restricted-master value after the stated number of iterations; on *I.16.15.2.90.k25* the pure-pool master is infeasible and the value is dominated by the big- M artificials until the pool can cover demand. Bottom: the rung inside the production runs of Table 7: pricing calls, calls the rung served without the exact oracle, columns it kept, exact-oracle calls, and the dispatched trucks of the reported plan (weighted by dispatch count) that are the rung’s columns.

<i>Measurement (results/fleet_tree_3d/ladder_measure_*.json)</i>						
Instance	mode	iterations	LP	columns	wall (s)	
<i>I_8.5.2.90</i>	exact oracle only	5	18 796.66	30	119.3	
<i>I_8.5.2.90</i>	rung, then oracle	5	18 798.56	48	126.4	
<i>I_16.15.2.90.k25</i>	exact oracle only	3	$3.40 \cdot 10^7$ (infeasible)	60	231.5	
<i>I_16.15.2.90.k25</i>	rung, then oracle	1		47	78.9	
<i>I_16.15.2.90.k25</i>	rung, then oracle	3		104	237.9	
<i>In the production runs (stats.ladder of the Table 7 files)</i>						
Instance	calls	served by rung	kept	exact calls	rung trucks in plan	
<i>I_8.5.2.90</i>	95	12	22	83	0 / 12	
<i>I_8.10.2.90.k25</i>	110	16	28	94	0 / 26	
<i>I_12.12.2.90.k25</i>	214	66	122	148	0 / 34	
<i>I_16.15.2.90.k25</i> , 1 800 s	609	210	364	399	41 / 54	
<i>I_16.15.2.90.k25</i> , 3 600 s	938	268	426	670	36 / 53	
<i>I_24.20.2.90.k50</i>	—	269	479	423	no plan	

Table 11: Fleet floors certified offline. The volumetric relaxation’s master, restricted to $\sum_{g,t} n_{gt} \leq X$, is priced to convergence with one thread (commit dd5f039); INFEASIBLE means no plan with at most X trucks exists, in 3D as well, since every placeable load is volumetrically feasible and the fleet row does not depend on the load model (Proposition 1). A bound instead of a refutation means the relaxation admits plans with X trucks, so nothing about the truck count is proved. *time* is the region certification alone; the root certificate it rests on took 4 to 32 s.

Instance	plan’s trucks	X tested	verdict	bound over region	time (s)
<i>I_8.5.2.90</i>	12	≤ 11	INFEASIBLE: 12 is the minimum fleet	—	1.5
<i>I_8.10.2.90.k25</i>	26	≤ 25	INFEASIBLE: 26 is the minimum fleet	—	3.9
<i>I_12.12.2.90.k25</i>	34	≤ 33	INFEASIBLE: 34 is the minimum fleet	—	16.3
<i>I_16.15.2.90.k25</i>	53	≤ 52	not refuted	68 131.43	1.3
<i>I_16.15.2.90.k25</i>	53	≤ 45	not refuted	68 131.43	1.2

Table 12: **[pending: regenerated benchmark table]** MATH-3D-SCPTOP against the volumetric benchmark on the 36 generated instances, under the budgets of Table 5. *loadable* is the outcome of the independent truck-by-truck verification; *unloadable* counts the SCPTOP loads a constraint programme proves cannot be placed, out of all loads it dispatches, with the undecided loads reported separately; Δz is the cost of MATH-3D-SCPTOP relative to SCPTOP.

Instance	MATH-3D-SCPTOP					SCPTOP (volumetric)					Δz
	z	trucks	LB (cert.)	loadable	time (s)	z	trucks	unloadable	undecided	time (s)	
[pending: 36 rows, one per instance]											

Table 13: **[pending: regenerated summary table]** Results by instance factor. *unloadable* is the share of SCPTOP truck loads proven impossible to place; *undecided* the share the proof could not settle within its limit.

Group	trucks (MATH)	trucks (SCPTOP)	unloadable	undecided	Δz	columns	time (s)
[pending: one row per level of N, T, R, v, and one for all]							

Table 14: **[pending: regenerated exact comparison]** MATH-3D-SCPTOP against the compact 3D-SCPTOP model on the small instances, the compact model warm-started with the matheuristic’s plan, both under the same wall-clock budget and the compact model under the declared memory cap. *stop* names the compact model’s stop condition; its LB is its dual bound at that stop and is valid whatever the stop condition. Rows marked *proved* are the only ones in which the last column is a gap to an optimum.

Instance	compact 3D-SCPTOP					MATH-3D-SCPTOP			
	z	LB	stop	vars/cons	time (s)	z	LB (cert.)	time (s)	Δz
[pending: one row per small instance]									

Table 15: **[pending: regenerated oracle comparison]** Pricing technology. The same method is run with the subproblem solved as a constraint programme (CP-SAT) and as a MILP with big- M non-overlap (Gurobi), under identical budgets; both oracles impose the full-truckload rule as the disjunction (42).

Instance	oracle	z	trucks	columns	LB (cert.)	time (s)
[pending: two rows per instance]						

Table 16: **[pending: case study not yet run against the current code]** The company baseline and the hand-drawn benchmark are rebuilt and solved to the same gap as the method before this table is regenerated.

Scenario	method	z	trucks	loadable	LB (cert.)	saving vs. company
[pending: three scenarios by three procedures]						

Table 17: **[pending: regenerated]** Measured solve time, compact 3D-SCPTOP against MATH-3D-SCPTOP, under an identical wall-clock budget. A method that stopped on its time or memory limit contributes that limit, not a solve time, and the stop condition is named.

instance	compact (s)	stop	MATH (s)	MATH/compact	Δz
[pending: one row per small instance]					

Table 18: **[pending: regenerated]** Measured solve time by approach. Times are wall-clock per instance; a method stopped by its limit contributes that limit, so medians at the limit indicate the budget, not convergence.

approach	set	n	median (s)	min	max
[pending: one row per approach and instance set]					

Table 19: **[pending: generated from the campaign now running]** The fleet tree (Algorithm 1, volumetric master) against the compact SCPTOP model on the size ladder, under the budgets of Table 5. Both solve the same problem. *stop* is the compact model’s stop condition and its LB the dual bound at that stop; the tree’s LB is its certified bound, with *root proved* recording whether the root sweep returned every pair proved. *open* is the number of regions the tree left unexpanded. No row claims a proof of optimality; the last column is the bracket in which the optimum lies.

compact SCPTOP					fleet tree								
Instance	z	LB	stop	time (s)	z	trucks	LB (cert.)	root proved	nodes	open	columns (priced / built)	time (s)	bracket
[pending: one row per ladder instance]													

6.5 Discussion

Head to head in 3D. Table 7 sets the compact 3D-SCPTOP model against MATH-3D-SCPTOP on three rungs of the ladder. On *I_8_5_2_90*, the one instance on which the compact model returns a plan, it has three configurations per direction, 67 674 variables and 92 092 constraints, and stops on its 1 800 s time limit at 18 816.13 with a dual bound of 18 795.76, no phantom freight, twelve loads that pass the independent verifier, and a peak resident set of 3.8 GB against the 20 GB cap. The fleet tree, from the ten single-commodity trucks and nothing else, constructs a twelve-truck root plan at 18 798.39 at 140.2 s and reaches 18 796.96 with twelve verified trucks at the first tree node, 185.4 s, which 146 further nodes fail to improve; its certified bound from the volumetric relaxation is 18 795.54, a gap of 0.0076%. Neither is an optimum, and the row says how each stopped. The two do *not* bracket the optimum, and the earlier revision of this section said they did. The compact model restricts each direction to $|L| = 3$ loading configurations for the whole horizon, and that restriction is chosen a priori: nothing here computes an over-estimate of $|L|$, and Section 4 states the equivalence only for an unrestricted $|L|$. A restriction’s optimum is at or above the problem’s, so its dual bound, 18 795.76, bounds the three-configuration model and not z^* . What the pair establishes is that both methods agree on twelve trucks and that the reformulation’s plan is 0.10% cheaper; the interval that holds z^* is the method’s own, [18 795.54, 18 796.96], a gap of 0.0076%. The region $X \leq 11$ was constructed in first and returned no plan over the pool, was priced under its own duals, and was left without a certificate inside the run; that eleven trucks is impossible is established offline (Table 11). The same plan was returned by the run on the preceding commit, before region certificates were taken from the volumetric relaxation, and by the same code with the 15% reserve (Table 9).

From ten periods on the compact model returns nothing. On *I_8_10_2_90_k25* — 119 424 variables and 164 692 constraints at $|L| = 3$ — the compact 3D model found *no feasible solution at all* in 1 800 s; on *I_12_12_2_90_k25*, with 200 748 variables and 256 180 constraints, and on *I_16_15_2_90_k25*, with 322 734 and 393 412, neither. All three stopped on time, not on memory: their peak resident sets were 6.1, 2.7 and 4.5 GB against the declared 20 GB cap, so the runs had their memory budget and did not use it, and the honest statement is that the model could not reach a first incumbent within the time it was given. Nor is this the leak guard rejecting a corrupt solution: *leaked* is zero because there was no solution to leak. Their dual bounds at the stop, 36 458.75, 48 516.86 and 64 053.23, are valid but lie 6.0%, 6.9% and 6.0% below the certified Dantzig–Wolfe bounds of 38 791.14, 52 105.25 and 68 131.43 (relative to those bounds; 6.4%, 7.4% and 6.4% relative to the compact model’s own). The reformulation therefore holds both sides at once on these rungs: the only feasible plans — twenty-six verified trucks at 38 855.70 in 900 s, thirty-four at 52 123.03 in 1 800 s, fifty-four at 78 808.39 in 1 800 s — and the better bounds, with certified gaps of 0.17% and 0.034% on the first two. On the ten-period instance the run on the commit preceding the final code, with the same cap, returned a plan 0.002% better, 38 855.06: the head-to-head carries the final-code row so that its three rows share one commit and one knob set, and the best verified plan on the instance is the earlier one. The twelve- and fifteen-period pairs are the equal-budget comparisons, 1 800 s on each side; on the ten-period pair the tree had half the compact model’s time. No optimum is known on any of them: the intervals [38 791.14, 38 855.06] and [52 105.25, 52 123.03] come from the reformulation alone, and the compact model contributes nothing to them. Read as a progression the ladder is short but clear: at five periods the compact model closes to 0.11% of its own bound and loses to a plan the tree had in hand nearly ten times sooner; from ten periods on it returns nothing. Four rungs are four rungs, and we do not generalise from them beyond the instance sizes they cover; the compact 3D model was not run at twenty periods.

The volumetric model as a cross-check. Every placeable load is volumetrically feasible and every column clears the full-truckload floor, so a verified 3D plan is a feasible solution of the volumetric problem: it can never be cheaper than the volumetric optimum, and the compact volumetric model’s dual bound, whatever its stop condition, is a valid bound for the 3D problem (Proposition 1). This is asserted, by a script that reads the result files and no solver, over every 3D row with a verified plan against every compact volumetric reference on the same instance, the rows added last included, and it holds on all of them; Table 8 reports the pairs. On *I_8_10_2_90_k25* the compact volumetric model, given 3 600 s, stopped with the status MEM_LIMIT at 3 243 s — the cap is per Gurobi model, and the process’s peak resident set was 21.5 GB — with an incumbent of 38 855.65 and a dual bound of 38 851.93; the best 3D plan, 38 855.06 in 900 s on the harder problem, lies above that dual bound, as it must, and below the incumbent, which it may while the compact model has not converged; and the volumetric bound is in fact the tighter valid 3D bound on this instance, placing that plan within 0.008% of its optimum and the final-code plan of 38 855.70 within 0.010%. Neither model proves anything here, and no optimum is claimed: what the pair shows is that on this

catalogue the geometry costs nothing measurable, and that the 0.17% gap the tree reports is the looseness of its own certificate, not of its plan.

When a volumetric capacity is adequate, and when it is not. Table 8 contains the finding that reframes the comparison. On *I_8.10.2.90.k25* the 3D plan matches the volumetric model’s twenty-six trucks and sits inside its interval; not one of the 428 distinct compositions the partition proposed was proved unplaceable. On *I_16.15.2.90.k25* the compact volumetric model is solved to *proven* optimality — forty-five trucks at 68 139.17 in 1 349 s — while the best verified 3D plan needs fifty-four trucks at 78 808.39, and 31.2% of the 984 distinct compositions proposed were proved unplaceable. The mechanism is in the catalogue. The error of a volumetric capacity is governed by whether a family’s height-limited and crush-limited layer counts coincide. On the first instance they do for both families (950 mm and 1 150 mm tall, three and two layers by either rule). On the second, the R_1 family is 650 mm tall — four layers under the 3 000 mm roof — but 378 to 484 kg full, two layers under the 1 200 kg crush limit for six of its eight products, so a volumetric fill credits a truck with roughly twice the placeable count of that family: the payload admits 57 full RTIs of P_1 where 32 can be placed. This cuts both ways, and is meant to. It says when a volumetric model is adequate — which most of the procurement transport literature assumes without checking — and it says when it is not, and the test is a property of the container catalogue that a planner can read off before choosing a model.

The gap on the crush-binding catalogue is in the bound. The 13.5% that Table 7 reports on *I_16.15.2.90.k25* is a gap to the certified volumetric bound, 68 131.43, and the same bound is loose for the same reason the volumetric plan is unplaceable: it credits every R_1 stack with four layers where two can be carried. Every other instance in the table measures under 0.2% against the same kind of bound, and we attribute the gap to the bound rather than to the plan; nothing in this paper can prove that, and the fifty-four-truck plan is reported as what it is, a verified upper bound under an 1 800 s budget. What can be said about the plan is how it was found. The restricted master over the seventeen single-commodity trucks is infeasible on this instance, at $1.55 \cdot 10^8$ of big- M artificials; three sweeps of a 5 s exact oracle leave it at $3.4 \cdot 10^7$, still not covering demand, while the heuristic rung brings it to a feasible 79 400 after one iteration and 76 810 after three (Table 10). In the 1 800 s run forty-one of the fifty-four dispatched trucks (thirty-eight of the forty-nine distinct loads) are the rung’s columns, and the tree opened no node: the root construction ran to the search deadline at 1 530 s, so this row is the construction’s plan over the rung’s columns and not yet the tree’s. With 3 600 s and integer stack counts in step 2 the root construction returns fifty-three verified trucks at 78 730.33 at 1 815 s, the tree opens the region $X \leq 52$ and finds no plan in it, and the run ends at 3 060 s with two regions open; the bound is unchanged, so the gap is still 13.5% and still the bound’s. Earlier configurations on this instance — a hand-seeded pool, a 4 GB cap, earlier commits and the earlier shelf packer — returned 88 255.57 with sixty-two trucks and 89 256.35 with sixty-six; those are earlier configurations, not a controlled comparison. A bound that is tight where the crush limit binds is sketched in the conclusions as the next step.

At twenty periods and five truckloads a period ($K = 21$) the primal does not scale. On *I_24.20.2.90.k50* — twenty-four products, twenty periods, a fleet of twenty-one — the method returned *no plan* in 3 600 s, and Table 8 records that as what it is, a measured limit. The bound side scaled: the volumetric relaxation certified 200 013.41 in 111 s, above the compact volumetric model’s dual bound of 199 518.04 at its memory stop, and it is the lower end of what the 3D optimum can be. The primal side did not, and the record says where the hour went. The pool of twenty-six single-commodity trucks took 11 s and root column generation its 530 s window (564 priced columns); step 2 returned a flow with $X = 154$ within its 300 s cap; and the partition then consumed 2 224 s — 557 s of partition MIPs and 1 667 s of placeability calls, 8 709 of them, 1 836 proved unplaceable and 192 undecided — on the plant pairs of the first fourteen of forty (direction, period) pairs, none realised at n_{gt} , before the search deadline cut the construction mid-loop; step 4 never ran. Memory was not binding, at a peak of 3.2 GB. What fails is the partition’s cut-and-repack walk on a crush-binding catalogue: step 2’s aggregate counts sit far from what can be placed truck by truck, so step 3 walks the placeability frontier 387 cut rounds long and does not converge. Since step 3’s deliverable is columns, not the partition — which the cover already conceded — a failing pair need not be pursued: keep the parts that packed, drop the rest, move to the next pair, and let step 4 combine what exists. That is a design change, proposed and not implemented, and we report the instance as unsolved. The 3D ladder therefore reads: certified gaps under 0.2% through twelve periods, a good plan with a loose bound at fifteen, and no plan at twenty periods with a fleet of twenty-one — against a compact 3D model that stops producing incumbents after five.

What the pool costs, and what the tree adds. Table 9 isolates two decisions on *I.8.5.2.90*. From the pure pool, with the equality region expanded first and a linear stack-area row in step 2, the tree returned thirteen trucks at 19 682.36 whether or not the rung was on: step 2 chose the twelve-truck flow at the root but asked the partition for a supplier delivery that needs thirty stacks of seven on a floor that holds twenty-nine, the pair went to two trucks, and the region $X \leq 12$ was never popped while the Murty subtree of $X = 13$ consumed the budget. Counting stacks in integers in step 2 and constructing $X \leq X^* - 1$ first returns the twelve-truck plan at 18 796.96 — to the cent the value of the hand-seeded reference, which ran at a different cap on an earlier commit and is not compared for time. The pure pool costs nothing here, and the reference it is measured against is the last hand-seeded number in the paper. What the tree then adds is bounded by what the root leaves it. On *I.8.10.2.90.k25* the root construction lands at 38 864.23 at 136.9 s and six Murty improvements take it to 38 855.70 at 481.9 s; the run on the preceding commit, from a root plan of 38 863.47 at 136.8 s, reached 38 855.06 at 535.9 s, 0.002% better, and the tree’s path depends on the regions it visits. On *I.12.12.2.90.k25* the root construction lands at 52 849.34 with thirty-four trucks at 530 s and six Murty improvements take it to 52 123.03 at 1 692 s. The same code with the 15% reserve (Table 9) shows the level-1 recursion at work on that instance: its root plan had thirty-five trucks, the region $X \leq 34$ returned a thirty-four-truck plan and was split on it, its piece $X \leq 33$ was priced and given up, and its piece $X = 34$ carried the incumbent to 52 129.91. On *I.8.5.2.90* the first tree node returns the best plan found. Pricing and branching are substitutes at very different prices, and the tree is the cheaper one where it has room. The frontier reserve itself is the one knob changed on the final code: with region certificates from the volumetric relaxation the sweep takes seconds, so at 15% the search stopped at 780 and 775 of 900 s and 1 561 of 1 800 s with the reserve unused, and at 3% the same code returns 38 855.70 and 52 123.03 against 38 859.44 and 52 129.91, and 18 796.96 again on *I.8.5.2.90*.

The pricing ladder, measured. The rung is qualified by reduced cost under the current duals, so what it contributes depends on the instance. Where the volumetric relaxation is nearly exact it is neutral to slightly negative: on *I.8.5.2.90* the stand-alone measurement of Table 10 reaches an LP of 18 798.56 after five iterations with the rung against 18 796.66 without it, on a pool sixty percent larger, and the production runs reach the same thirteen-truck plan at 19 682.36 with or without it (Table 9). On *I.16.15.2.90.k25* it is the difference between a feasible master and none. Inside the production runs it served 16, 16, 66, 210 and 268 of the 101, 110, 214, 609 and 938 pricing calls without the exact oracle, and its share of the dispatched trucks rises from none on the three smallest instances to forty-one of fifty-four and thirty-six of fifty-three on *I.16.15.2.90.k25*; on *I.24.20.2.90.k50* it served 269 pairs and no plan followed. The certificate is never the rung’s: the bound of every row is one uncapped exact sweep of the volumetric oracle.

Where the time goes in 3D. It depends on whether the crush limit binds. On *I.8.10.2.90.k25* the aggregate master of step 2 took 486 s of an 883 s run, root column generation 179 s, the partitions 199 s nearly all of them placeability calls, and the integer master 7 s; the aggregate MIP is the scaling limit there, and it is shared with the volumetric method. On *I.12.12.2.90.k25* step 2 took 311 s of a 1 770 s run and the partitions 901 s, with 847 s of the latter in placeability. On *I.16.15.2.90.k25* at 1 800 s step 2 took 4 s and the partitions 1 174 s of 1 530, of which 920 s were placeability calls, and at 3 600 s the partitions took 1 878 s of 3 060 with 1 737 s in placeability; on *I.24.20.2.90.k50* the partition absorbed 2 224 s of 3 065 and the run ended without a plan. Where the volumetric relaxation proposes parts that cannot be placed, proving so is where the time goes, and the geometry is no longer nearly free. The constructive shelf design answered 1 653 of the 1 682 successful placements on the 1 800 s run of *I.16.15.2.90.k25* and the constraint programme the other 29, together with the 307 proofs of infeasibility.

Bracketing the volumetric optimum. On the size ladder the fleet tree over the volumetric oracle and the compact SCPTOP model solve the same problem, so each returns a plan and a bound for the other. The compact references are recorded: with 3 600 s, ten threads and a 20 GB cap the compact model proves *I.16.15.2.90.k25* optimal and stops on its memory cap on *I.8.10.2.90.k25*, *I.16.15.2.90.k50*, *I.24.20.2.90.k50* and *I.24.20.2.90.k80*, keeping an incumbent and a dual bound in each case. The volumetric fleet-tree rows on those instances predate the recording of a configuration with every run and are not reported; Table 19 is regenerated from runs that carry one, and no bracket is quoted until it is.

The truck counts are proved through twelve periods. The region $X \leq X^* - 1$ asks whether a plan with fewer trucks exists, and the volumetric relaxation answers it in seconds: its master restricted to $\sum_{g,t} n_{gt} \leq X$ and priced to convergence either returns a bound or comes back infeasible, and an infeasible region is infeasible in 3D as well, because every placeable load is volumetrically feasible and the fleet row

does not depend on the load model (Proposition 1). Table 11 reports these certificates, obtained offline with one thread. On *I_8.5.2.90*, *I_8.10.2.90.k25* and *I_12.12.2.90.k25* the regions $X \leq 11$, $X \leq 25$ and $X \leq 33$ are *infeasible*, in 1.5, 3.9 and 16.3 s, so twelve, twenty-six and thirty-four trucks are the minimum fleets and the reported plans have them: what remains of the gaps — 0.0076%, 0.17% and 0.034% to the LP-certified bounds, and 0.0064% and 0.010% to the tighter compact bounds on the first two — is inventory cost. On *I_16.15.2.90.k25* neither $X \leq 52$ nor $X \leq 45$ is refuted: the bound over either region is 68131.43, the root value, because the volumetric relaxation admits forty-five-truck plans there — the compact volumetric optimum has forty-five — and so cannot separate the fifty-three-truck 3D plan from fifty-two; nothing about the truck count is proved on that instance, for the same reason the bound is loose. No run of the night made these statements itself: the tree’s certification of $X \leq X^* - 1$ was gated on the geometric pool LP, which never allowed it, a defect fixed after the runs and not re-run; the offline certificate is the honest source.

What the tree does and does not deliver. On the one-commit campaign of Table 6 the tree leaves *no region uncovered*: *dropped* is zero on all five instances, against 86, 18 and 8 on the previous revision. That is a real change and it is not the same as an exhaustive search, so we state the difference precisely. A region is *closed* here when it carries a certified bound; it is *refuted* only when that bound reaches the incumbent or the region is proved empty. Classifying every region *I_8.5.2.90* popped: 24 internal (split into disjoint, exhaustive children), 15 leaves proved infeasible by certification, 12 leaves whose fleet polytope is empty by arithmetic, 7 leaves pruned on a certified bound — and 3 leaves that are neither. Those three fix the *entire* fleet vector, so the tree has nothing left to branch on, and the construction that explored them is a primal heuristic. Three regions, therefore, stand between that run and an exhaustive search, and the run’s *proved* flag correctly refuses.

We tried to close them and failed. Certifying each of the three under its own duals — one exact sweep of $2|T|$ pricing calls, 237 s apiece — returned 18795.57, 18795.54 and 18795.54 against an incumbent of 18796.96, none of them proved. The reason is arithmetic rather than accidental: with the fleet fixed at twelve trucks, refuting a leaf needs a maximum reduced cost below roughly 0.12 per column, while pricing on the same instance was still returning columns an order of magnitude below that. Meanwhile the volumetric relaxation of the same instance solves to proved optimality in 0.18 s, at exactly the cost of the verified 3D plan, which settles the question the certification could not (Table 6, footnote *b*). We report that contrast as we found it: on this instance the bound the method needs is available from the relaxation and not from its own machinery.

The tree’s contribution is therefore uneven across the ladder and we do not average it away. At five and ten periods it opens 67 and 68 nodes and prunes 7 and 13 regions on certified bounds. At twelve periods it opens 11; at fifteen and twenty it opens 1 and 2, and on those two rows the plan is the root construction’s — the improvement of 0.33% on *I_16.15.2.90.k25* and the first plan ever obtained on *I_24.20.2.90.k50* are the construction’s results, not the search’s. Finally, in every 3D run the global bound equals the root bound, because a region’s certificate is taken from the volumetric relaxation and no such certificate can exceed the relaxation’s optimum. Where that optimum coincides with the 3D optimum the method can prove optimality; where the crush limit binds and it does not, the residual gap is not closable by more search, and the crush-aware stack-count rows of Section 5.7 are what the bound would need.

A volumetric capacity is not a conservative approximation. The loads that SCPTOP dispatches are each submitted to a placeability proof. The share proved impossible to place, by instance factor, is reported in Table 13, together with the share left undecided, which must be read alongside it: a zero count of proven infeasibilities means nothing if most loads were undecided. The direction of the error is what the comparison is designed to expose. A volumetric capacity ignores the remainders that footprints and layer heights leave unusable, so where it errs it errs by dispatching loads that do not fit, and a planner handed such a plan discovers the shortfall on the loading dock. Whether that error concentrates on the coarse containers, as the geometry suggests it should, is a reading of Table 13 and is left to the regenerated table.

Feasibility by construction. Every plan the geometric method returns is re-verified truck by truck against (38)–(43) by code that shares nothing with the solver that produced it, and the *loadable* column of Table 12 records the outcome for every row. That is a property of the formulation rather than of the search: a column is a witnessed layout, so a plan assembled from columns is executable by construction and can be handed to a loading crew as a drawing. The realised utilisation of the least-filled truck in each plan is reported with the table.

Against the procedure in use. The case study of Section 7 compares the same machinery against the company’s average-mix configurations and against a richer hand-drawn set. Its table is pending regeneration with the current code and with the baselines solved to the same gap as the method, and no percentage from the previous revision is repeated here: the company’s configurations in the existing file were laid out by an earlier, less dense shelf packer, so the baseline there is artificially worse and the saving over it overstated.

Pricing technology. The subproblem was solved both as a constraint programme and as a MILP with the big- M non-overlap encoding (16)–(19), by running the whole method with each backend on the same instances under identical budgets (Table 15). The comparison is valid only because both oracles now solve the same problem: an earlier version of the MILP oracle imposed the full-truckload requirement as the sum of the two ratios rather than as the disjunction (42), solved a strictly weaker problem, and produced a column that failed verification, so the advantage it appeared to show was an artefact. Which technology is used for the geometric rows, and on what evidence, is stated with the regenerated table.

Where each method belongs. Table 14 sets the compact 3D-SCPTOP model against the matheuristic on the small instances, both under the same wall-clock budget and the compact model warm-started with the matheuristic’s plan. The reading we expect, and will state only where the table supports it, is that the crossover is the planning horizon and the fleet rather than the raw size of the formulation: where the compact model closes an instance it remains the better tool, and the case for the decomposition rests on the regime beyond that, where the compact model’s tree grows to its memory budget with a stalled bound while the decomposition returns a verified plan and a certified bound. The size ladder is where that regime is measured.

7 Practical application

This section presents a practical procurement transport application. The proposed resolution framework was evaluated using real data from an automotive industry supply chain. We validate and compare the 3D-SCPTOP solution procedures to the company’s procedure, which employs predefined loading configurations that indicate the number of each container type to be carried. Finding these loading configurations is a very tedious and time-consuming manual task that is based on a spreadsheet procedure. A mathematical programming model, dubbed as PLC-SCPTOP, which replicates the current’s company procedure, is then utilised to minimise the total cost, with a summation of procurement transport and inventory holding costs, and one based on these predetermined configurations.

The real-world data derived from the company’s operations act as the empirical foundation for the comparison. To broaden its scope we scale and perturb these data under a stochastic process, giving three scenarios that differ only in demand variability: low ($\pm 5\%$), normal ($\pm 15\%$) and high ($\pm 35\%$) around the same mean. Three procedures are compared on the same instances.

- **Company procedure.** PLC-SCPTOP over the configurations the planner derives today: a truck laid out to follow the *average* container mix across plants and periods, recomputed as the programme is revised, together with one standard single-container-type pattern per RTI type and the partial loads a planner will dispatch rather than miss a delivery.
- **Hand-drawn benchmark.** PLC-SCPTOP over a richer predefined set: a regular grid per container type plus two-zone layouts in which the floor is cut once and each zone tiled with a single footprint. This is more than the planner actually maintains, and is therefore the harder baseline.
- **MATH-3D-SCPTOP** of Section 5, which generates its configurations and augments the planner’s existing ones rather than replacing them.

Table 16 reports the outcome. **[pending: The case study has not yet been run against the current code; the baselines must be solved to the same gap as the method before any saving is stated, and the percentages of the previous revision are not repeated here.]** features of the comparison matter more than the percentages, and both are readings of the regenerated table rather than claims made in advance of it.

First, whether the advantage erodes as demand variability grows. The expected behaviour of a fixed configuration set is the opposite: an average mix tracks a varying programme progressively worse, and the gap it leaves is exactly what generated configurations recover. If the table bears that out, it also answers the concern that such a method should help only when configurations and demand are misaligned — the misalignment would then be the normal state of a programme that varies at all, not an occasional pathology.

Second, the size of the configuration set matters as much as its content. Reconstructing the company’s set from average mixes alone yields very few distinct configurations when demand is stable — at low variability the horizon average, the thirds and the per-period mixes all coincide — and a set that small may not serve the programme. Crews do not work that way: they accumulate patterns by trial, one for the days a single product runs heavy, another for when two share the load. The number of configurations the reconstruction needs before the programme can be served is itself informative, and is reported with the table (**[pending: configuration counts by scenario]**).

That is the practical cost of a fixed configuration set. It is not that the procedure fails — the plant plainly operates — but that keeping it feasible as the programme moves requires the planner to maintain a large and growing catalogue by hand, and every configuration in it was found by trial rather than by design.

The managerial reading is that the benefit is not a better spreadsheet. It is that the loading configuration stops being an input fixed months in advance and becomes a decision taken with the transport plan, so that when the container mix moves the configurations move with it.

The method is therefore designed to reduce inventory and transport costs precisely where a fixed configuration set is weakest: under variable demand and misalignment between loading configurations and container types. The regenerated case-study table is what will say whether it does.

8 Conclusions

We herein propose a MILP model, 3D-SCPTOP, to tackle procurement transportation planning at an operational level by incorporating stack-based full-truck loading constraints into its considerations. Given the NP-hard nature of the placement subproblem and the fact that the compact model replicates it once per loading configuration, we reformulate the problem by Dantzig–Wolfe decomposition, taking the complete loaded truck as the unit of decision, and solve it by a branch and bound over the fleet vector with column generation, a constraint programming pricing oracle and a flow-first construction as primal heuristic, giving MATH-3D-SCPTOP. Because every column is a fully specified truck layout, the plans the method returns are executable by construction and can be handed to a loading crew as a drawing; we verify each dispatched truck independently of the solver that produced it.

A case study has also been conducted within an automotive SC that comprises a car assembler, a first-tier supplier and a second-tier supplier. The objective of the 3D-SCPTOP model is twofold: (i) to minimise the total number of trucks used; (ii) to minimise inventory holding costs. This approach takes into account the complexities of supply chain logistics in the automotive industry by demonstrating a successful integration of procurement transport planning and 3D loading constraints.

Managerial implications in a SC context are oriented to the proposed solution methodology, which is designed to improve on the solutions found by the company’s procedure and to adapt to demand changes while returning executable plans in reasonable computational times; the case-study comparison that measures this is being regenerated under declared budgets.

A second managerial point concerns the representation of transport capacity. Procurement transport models in the literature typically express it volumetrically. Our comparison is built to show that doing so is not a harmless simplification: a volumetric model can return plans that are cheaper on paper precisely because truck loads it prescribes cannot be placed in a trailer at all, and each such load is submitted to an independent placeability proof so that the share is measured rather than asserted. The measurements attach a mechanism to the claim: where a container family’s height-limited and crush-limited layer counts coincide the volumetric model is adequate, and where they do not it credits a truck with roughly twice the placeable count of that family and its optimum is not a reachable target. A planner acting on such a plan discovers the shortfall on the loading dock, not in the model.

Nevertheless, several challenges remain for further research lines to deal with; for instance, the inclusion of multiple suppliers to better represent complex supply chains. A more comprehensive transport operation model could be achieved by incorporating routing aspects, such as vehicle routing with 3D loading constraints. Future work may also relax the layer-homogeneity assumption to allow intra-layer mixing of RTI types, or extend the placement block with axle load-balance constraints. On the algorithmic side, the fleet tree is an exact scheme whose bound is certified at the root and at the one region that could refute the incumbent, and in no run reported here did it close its gap: the cost of certifying every node with a geometric pricing oracle is what stands between the matheuristic and a proof, and a cheaper certificate for the constraint programme is the natural next step. The construction under 3D loading keeps the volumetric partition as a relaxation and cuts off the parts a placeability proof refutes; where the volumetric relaxation is nearly exact its cost is dominated by the aggregate integer masters it shares with the volumetric method, and where the crush limit binds it is dominated by the placeability proofs, so both are where scaling effort

belongs. The bound is the other open end. On the catalogue where the crush limit halves a family's layer count the certified volumetric bound sits 13.5% below the best verified plan, a gap we attribute to the bound and not to the plan, since every other instance measures under 0.2%; and the remedy is already in the method's own rows. The stack-count relaxation that step 2 and the partition use — an integer number of stacks per footprint, each holding at most $W^{\max}$ of weight and rK of height, their floor areas summed against the trailer floor — is a necessary condition for any placeable load under the crush rule (41). Its continuous crush row, $\sum_f \text{area}_f (\sum_{c \in f} w_c a_c) / W^{\max} \leq pK \cdot qK$, is a third knapsack resource for the volumetric pricing oracle that certifies the reported bound, and for a footprint carried by a single RTI type an integer-layer height row, $\sum_{c \in f} a_c \leq \lfloor rK/h_f \rfloor s_f$, joins it. The per-commodity form $\sum_c (\text{area}_c / \text{layers}_c) a_c$ is *not* valid where products of different binding layer counts share a footprint: one heavy crush-limited layer under two light height-limited ones is a legal stack that the row charges $1/2 + 2/3$ of a stack, and it is exactly the mixed stack the heuristic rung builds on that catalogue. In the column master the rows are implied, since every column is placeable; they bite only in the relaxation, and on a catalogue where crush does not bind they change nothing. We record this as the next step, with its validity argument and without a result. The other limit is the primal at scale: on the twenty-period instance the partition's cut-and-repack walk consumed the hour on a third of the pairs and no plan was built, while the pool, the pricing and the certificate all scaled. Step 3 exists to generate columns, not to realise a partition, so a pair whose parts do not place need not be pursued round after round; capping its rounds, keeping the parts that placed and letting the integer master combine whatever columns exist is the change to measure next, and it is proposed here and not implemented. Integrating sourcing and delivery transport decisions could improve supply chain efficiency. Finally, the incorporation of production scheduling into the model has the potential of offering a cohesive approach to supply chain planning optimisation by aligning production timelines with transportation and inventory strategies.

A The SCPTOP benchmark

The benchmark against which 3D-SCPTOP is compared is an extension of the procurement transport model of (Diaz-Madroño, Mula, and Peidro 2014) to the closed-loop, two-node setting studied here. Three modifications are made to that model, as stated in Section 6: fuzzy parameters are dropped; the transport capacity restriction is expressed through the volume of the individual RTIs against the truck volume rather than a maximum container count; and a product may travel in more than one RTI type.

The point of the comparison is to isolate the effect of the loading representation, so SCPTOP is made *identical to the transport and inventory block of 3D-SCPTOP in every other respect*: the same two nodes, the same closed loop of full and folded RTIs, the same lead time, the same coverage stock, the same payload cap, the same full-truckload floor and the same objective. It differs in exactly one place — where 3D-SCPTOP asks whether the RTIs can be stacked and placed on the trailer floor, SCPTOP asks only whether their volumes sum to less than the trailer's.

Let $Y_{gkt} \in \{0, 1\}$ indicate that truck k is used in direction g in period t , and let Q_{nt}^k and Q_{rt}^k be the full RTIs of product n , respectively the folded RTIs of type r , that it carries. All remaining notation is that of Table 3.

$$\text{Min } z = \sum_{g \in G} \sum_{k \in K} \sum_{t \in T} cK Y_{gkt} + \sum_{g \in G} \sum_{n \in N} \sum_{t \in T} ic_{gn} I_{gnt} \quad (69)$$

subject to, for all $k \in K$ and $t \in T$,

$$\sum_{n \in N} \gamma_{pp(n)} Q_{nt}^k \leq \Gamma_K Y_{pkt}, \quad \sum_{r \in R} \gamma_{sr} Q_{rt}^k \leq \Gamma_K Y_{skt} \quad (70)$$

$$\sum_{n \in N} w_n Q_{nt}^k \leq pK^w Y_{pkt}, \quad \sum_{r \in R} w_r^f Q_{rt}^k \leq pK^w Y_{skt} \quad (71)$$

together with the full-truckload floor on the binding resource, imposed exactly as in (42): a dispatched truck must reach a fraction α of *either* the trailer volume or the payload,

$$\sum_n \gamma_{pp(n)} Q_{nt}^k \geq \alpha \Gamma_K (Y_{pkt} - 1 + \theta_{kt}), \quad \sum_n w_n Q_{nt}^k \geq \alpha pK^w (Y_{pkt} - \theta_{kt}), \quad (72)$$

with $\theta_{kt} \in \{0, 1\}$ and the supplier direction analogous. The flows aggregate as $Q_{nt} = \sum_k Q_{nt}^k$ and $Q_{rt} = \sum_k Q_{rt}^k$, and the inventory balances, coverage stock and domains are those of (2)–(12), unchanged.

Constraint (70) is the whole of the benchmark’s loading model: a truck is deemed able to carry any set of RTIs whose volumes fit inside the trailer’s. No footprint, stack height, layer count or crush limit appears. Section 6 reports what that omission costs — not in solution quality, where SCPTOP appears cheaper, but in whether the plans it prescribes can be loaded at all.

Why the floor is imposed on the binding resource. Writing (72) on volume alone, as the original formulation does, makes dense freight structurally infeasible: filling α of a 97.9 m^3 trailer with RTIs at 590 kg/m^3 weighs 27.4 t against a 24 t cap, so no truck is dispatchable and the instance is empty. Summing the two ratios instead is strictly weaker — it accepts a truck 35% full by volume and 20% by weight. The disjunctive form above is used identically in every model compared here, so no method is held to an easier standard than another.

References

- Ali, Sara et al. (June 2022). “On-line three-dimensional packing problems: A review of off-line and on-line solution approaches”. In: *Computers and Industrial Engineering* 168. ISSN: 03608352. DOI: 10.1016/j.cie.2022.108122.
- Alonso, M. T., R. Alvarez-Valdes, and F. Parreño (Mar. 2020). “A GRASP algorithm for multi container loading problems with practical constraints”. In: *4OR* 18.1, pp. 49–72. ISSN: 16142411. DOI: 10.1007/s10288-018-0397-z.
- Alonso, M. T., A. Martinez-Sykora, et al. (Nov. 2022). “The pallet-loading vehicle routing problem with stability constraints: The pallet-loading vehicle routing problem with stability constraints. Second revision”. In: *European Journal of Operational Research* 302.3, pp. 860–873. ISSN: 03772217. DOI: 10.1016/j.ejor.2022.01.035.
- Baller, Reinhard et al. (July 2022). “Optimizing automotive inbound logistics: A mixed-integer linear programming approach”. In: *Transportation Research Part E: Logistics and Transportation Review* 163. ISSN: 13665545. DOI: 10.1016/j.tre.2022.102734.
- Berman, Oded and Qian Wang (2006). “Inbound logistic planning: Minimizing transportation and inventory cost”. In: *Transportation Science* 40.3, pp. 287–299. ISSN: 15265447. DOI: 10.1287/trsc.1050.0130.
- Bortfeldt, Andreas and Gerhard Wäscher (Aug. 2013). “Constraints in container loading-A state-of-the-art review”. In: *European Journal of Operational Research* 229.1, pp. 1–20. ISSN: 03772217. DOI: 10.1016/j.ejor.2012.12.006.
- Chen, C S, S M Lee, and Q S Shen (1995). *EUROPEAN JOURNAL OF OPERATIONAL RESEARCH An analytical model for the container loading problem*. Tech. rep., pp. 68–76.
- Chen, Mingzhou, Jiazhen Huo, and Yongrui Duan (June 2023). “A hybrid biogeography-based optimization algorithm for three-dimensional bin size designing and packing problem”. In: *Computers and Industrial Engineering* 180. ISSN: 03608352. DOI: 10.1016/j.cie.2023.109239.
- Chitsaz, Masoud, Jean François Cordeau, and Raf Jans (Dec. 2019). “A unified decomposition matheuristic for assembly, production, and inventory routing”. In: *INFORMS Journal on Computing* 31.1, pp. 134–152. ISSN: 15265528. DOI: 10.1287/ijoc.2018.0817.
- Chopra, Sunil and Peter Meindl (2015). *Supply chain management : strategy, planning, and operation*. ISBN: 9780133800203.
- Díaz-Madroño, Manuel, Josefa Mula, and David Peidro (Sept. 2014). *A review of discrete-time optimization models for tactical production planning*. DOI: 10.1080/00207543.2014.899721.
- (Dec. 2017). “A mathematical programming model for integrating production and procurement transport decisions”. In: *Applied Mathematical Modelling* 52, pp. 527–543. ISSN: 0307904X. DOI: 10.1016/j.apm.2017.08.009.
- Díaz-Madroño, Manuel, David Peidro, and Josefa Mula (Dec. 2014). “A fuzzy optimization approach for procurement transport operational planning in an automobile supply chain”. In: *Applied Mathematical Modelling* 38.23, pp. 5705–5725. ISSN: 0307904X. DOI: 10.1016/j.apm.2014.04.053.
- (Oct. 2015). *A review of tactical optimization models for integrated production and transport routing planning decisions*. DOI: 10.1016/j.cie.2015.06.010.

- Dominguez, Oscar et al. (Jan. 2016). “Using biased randomization for solving the two-dimensional loading vehicle routing problem with heterogeneous fleet”. In: *Annals of Operations Research* 236.2, pp. 383–404. ISSN: 15729338. DOI: 10.1007/s10479-014-1551-4.
- Dyckhoff, Harald (1990). *A typology of cutting and packing problems*. Tech. rep., pp. 145–159.
- Fleischmann, Bernhard, Herbert Kopfer, and Christopher Sürie (Mar. 2015). “Transport Planning for Procurement and Distribution”. In: *Supply Chain Management and Advanced Planning*. Ed. by Hartmut Stadtler, Christoph Kilger, and Herbert Meyr. Springer Texts in Business and Economics. Springer. Chap. 12, pp. 225–240. DOI: 10.1007/978-3-642-55309-7. URL: https://ideas.repec.org/h/spr/sptchp/978-3-642-55309-7_12.html.
- Gonçalves, José Fernando and Mauricio G.C. Resende (Oct. 2013). “A biased random key genetic algorithm for 2D and 3D bin packing problems”. In: *International Journal of Production Economics* 145.2, pp. 500–510. ISSN: 09255273. DOI: 10.1016/j.ijpe.2013.04.019.
- Guo, Min et al. (2024). “Incentivising-by-penalty: The optimal return strategy for a reusable transport item rental platform”. In: *Transportation Research Part D: Transport and Environment* 134, p. 104339. ISSN: 1361-9209. DOI: <https://doi.org/10.1016/j.trd.2024.104339>. URL: <https://www.sciencedirect.com/science/article/pii/S1361920924002967>.
- Harrath, Youssef (Sept. 2022). “A three-stage layer-based heuristic to solve the 3D bin-packing problem under balancing constraint”. In: *Journal of King Saud University - Computer and Information Sciences* 34 (8), pp. 6425–6431. ISSN: 22131248. DOI: 10.1016/j.jksuci.2021.07.007.
- Hashemi-Amiri, Omid, Fahimeh Ghorbani, and Ran Ji (Jan. 2023). “Integrated supplier selection, scheduling, and routing problem for perishable product supply chain: A distributionally robust approach”. In: *Computers and Industrial Engineering* 175. ISSN: 03608352. DOI: 10.1016/j.cie.2022.108845.
- Hein, Fanny and Christian Almeder (July 2016). “Quantitative insights into the integrated supply vehicle routing and production planning problem”. In: *International Journal of Production Economics* 177, pp. 66–76. ISSN: 09255273. DOI: 10.1016/j.ijpe.2016.04.014.
- Junqueira, Leonardo, Reinaldo Morabito, and Denise Sato Yamashita (2012a). “MIP-based approaches for the container loading problem with multi-drop constraints”. In: *Annals of Operations Research* 199.1, pp. 51–75. ISSN: 15729338. DOI: 10.1007/s10479-011-0942-z.
- (Jan. 2012b). “Three-dimensional container loading models with cargo stability and load bearing constraints”. In: *Computers and Operations Research* 39.1, pp. 74–85. ISSN: 03050548. DOI: 10.1016/j.cor.2010.07.017.
- Junqueira, Leonardo, José F. Oliveira, et al. (Sept. 2013). “An optimization model for the vehicle routing problem with practical three-dimensional loading constraints”. In: *International Transactions in Operational Research* 20.5, pp. 645–666. ISSN: 09696016. DOI: 10.1111/j.1475-3995.2012.00872.x.
- Kurpel, Deidson Vitorio et al. (July 2020). “The exact solutions of several types of container loading problems”. In: *European Journal of Operational Research* 284.1, pp. 87–107. ISSN: 03772217. DOI: 10.1016/j.ejor.2019.12.012.
- Langley, C. John et al. (2020). *Supply Chain Management: A Logistics Perspective*. 11th. Cengage Learning, pp. 413–452.
- Li, Ying, Mingzhou Chen, and Jiazhen Huo (Mar. 2022). “A hybrid adaptive large neighborhood search algorithm for the large-scale heterogeneous container loading problem”. In: *Expert Systems with Applications* 189. ISSN: 09574174. DOI: 10.1016/j.eswa.2021.115909.
- Mahvash, Batoul, Anjali Awasthi, and Satyaveer Chauhan (2018). “A column generation-based heuristic for the three-dimensional bin packing problem with rotation”. In: *Journal of the Operational Research Society* 69 (1), pp. 78–90. ISSN: 14769360. DOI: 10.1057/s41274-017-0186-7.
- Meyer, Anne and Boris Amberg (May 2018). “Transport concept selection considering supplier milk runs – An integrated model and a case study from the automotive industry”. In: *Transportation Research Part E: Logistics and Transportation Review* 113, pp. 147–169. ISSN: 13665545. DOI: 10.1016/j.tre.2017.07.004.
- Mosca, Alyssa, Navneet Vidyarthi, and Ahmet Satir (Jan. 2019). *Integrated transportation – inventory models: A review*. DOI: 10.1016/j.orp.2019.100101.

- Paquay, C, M Schyns, and S Limbourg (2016). “INTERNATIONAL TRANSACTIONS IN OPERATIONAL RESEARCH A mixed integer programming formulation for the three-dimensional bin packing problem deriving from an air cargo application”. In: *Intl. Trans. in Op. Res* 23, pp. 187–213. DOI: 10.1111/itor.12111. URL: <https://onlinelibrary.wiley.com/doi/10.1111/itor.12111>.
- Que, Quanqing, Fang Yang, and Defu Zhang (Mar. 2023). “Solving 3D packing problem using Transformer network and reinforcement learning”. In: *Expert Systems with Applications* 214. ISSN: 09574174. DOI: 10.1016/j.eswa.2022.119153.
- Sancak, Emre and F. Sibel Salman (June 2011). “Multi-item dynamic lot-sizing with delayed transportation policy”. In: *International Journal of Production Economics* 131.2, pp. 595–603. ISSN: 09255273. DOI: 10.1016/j.ijpe.2011.02.004.
- Schöneberg, Tim, Achim Koberstein, and Leena Suhl (Dec. 2010). “An optimization model for automated selection of economic and ecologic delivery profiles in area forwarding based inbound logistics networks”. In: *Flexible Services and Manufacturing Journal* 22.3-4, pp. 214–235. ISSN: 19366582. DOI: 10.1007/s10696-011-9084-5.
- (Nov. 2013). “A stochastic programming approach to determine robust delivery profiles in area forwarding inbound logistics networks”. In: *OR Spectrum* 35.4, pp. 807–834. ISSN: 01716468. DOI: 10.1007/s00291-013-0349-0.
- Silva, Everton Fernandes, Túlio Angelo Machado Toffolo, and Tony Wauters (Sept. 2019). “Exact methods for three-dimensional cutting and packing: A comparative study concerning single container problems”. In: *Computers and Operations Research* 109, pp. 12–27. ISSN: 03050548. DOI: 10.1016/j.cor.2019.04.020.
- Soysal, Mehmet et al. (2023). “Managing returnable transport items in a vendor managed inventory system”. In: *Socio-Economic Planning Sciences* 86, p. 101504. ISSN: 0038-0121. DOI: <https://doi.org/10.1016/j.seps.2022.101504>. URL: <https://www.sciencedirect.com/science/article/pii/S0038012122003111>.
- Stadtler, Hartmut, Christoph Kilger, and Herbert Meyr (2015). *Supply Chain Management and Advanced Planning*. Vol. 5th Edition. URL: <http://www.springer.com/series/10099>.
- Tresca, Giulia et al. (July 2022). “Automating Bin Packing: A Layer Building Matheuristics for Cost Effective Logistics”. In: *IEEE Transactions on Automation Science and Engineering* 19 (3), pp. 1599–1613. ISSN: 15583783. DOI: 10.1109/TASE.2022.3177422.
- Wäscher, Gerhard, Heike Haußner, and Holger Schumann (Dec. 2007). “An improved typology of cutting and packing problems”. In: *European Journal of Operational Research* 183.3, pp. 1109–1130. ISSN: 03772217. DOI: 10.1016/j.ejor.2005.12.047.
- Wei, Lijun et al. (June 2015). “A variable neighborhood search for the capacitated vehicle routing problem with two-dimensional loading constraints”. In: *European Journal of Operational Research* 243.3, pp. 798–814. ISSN: 03772217. DOI: 10.1016/j.ejor.2014.12.048.
- Zhang, Yipei et al. (2024). “Closed-loop inventory routing problem for perishable food with returnable transport items selection”. In: *International Journal of Production Research* 62.1-2, pp. 501–521. DOI: 10.1080/00207543.2023.2275639. eprint: <https://doi.org/10.1080/00207543.2023.2275639>. URL: <https://doi.org/10.1080/00207543.2023.2275639>.
- Zhao, Hang, Chenyang Zhu, and Hui Huang (2022). “Learning practically feasible policies for online 3D bin packing”. In: 65.112105, p. 17. DOI: 10.1007/s11432-021-3348-6. URL: <https://doi.org/10.1007/s11432-021-3348-6>.
- Zhao, Xiaozhou et al. (2016). “INTERNATIONAL TRANSACTIONS IN OPERATIONAL RESEARCH A comparative review of 3D container loading algorithms”. In: *Intl. Trans. in Op. Res* 23, pp. 287–320. DOI: 10.1111/itor.12094. URL: <https://onlinelibrary.wiley.com/doi/10.1111/itor.12094>.
- Zhu, Wenbin and Andrew Lim (Nov. 2012). “A new iterative-doubling Greedy-Lookahead algorithm for the single container loading problem”. In: *European Journal of Operational Research* 222.3, pp. 408–417. ISSN: 03772217. DOI: 10.1016/j.ejor.2012.04.036.